國 立 中 正 大 學

資訊工程研究所碩士論文

基於整數化量化的 CNN 加速器在 FPGA 上進行即時臉部表情分析

Real-Time Facial Expression Analysis on FPGA Using an Integer-Only Quantized CNN Accelerator

研 究 生 ： 黃鈺翔

指導教授 ： 鍾菁哲 博士

中華民國 一一四 年 八 月

國立中正大學碩士學位論文考試審定書

資訊工程學系

研究生 黃鈺翔 所提之論文

Real-Time Facial Expression Analysis on FPGA Using an Integer-Only Quantized CNN Accelerator 基於整數化量化的 CNN 加速器在FPGA上進行即時臉部表情分析

經本委員會審查，符合 碩 士學位論文標準。

學位考試委員會
召集人 李順裕 簽章

委員 李順裕 鍾雲哲 成鍔 黃崇勳

指導教授 鍾雲哲 簽章

中華民國 114 年 7 月 3 日

摘要

臉部表情辨識（Facial Expression Recognition, FER）在智慧醫療、駕駛監控與人機互動等邊緣運算應用中愈加重要。然而，深度學習模型在如 FPGA 等資源受限平台上的部署面臨計算能力與功耗限制的重大挑戰。本論文提出一套以整數化卷積神經網路為核心的即時 FER 系統，成功實作於 FPGA 硬體上。

本研究設計一個精簡且量化的 MobileNetV2 模型，處理 100×100 的輸入影像，兼顧準確率與運算效率。模型訓練採用具抗噪能力的 Erasing Attention Consistency（EAC）策略，以提升於自然場景資料集上的辨識效能。經過每通道對稱量化與殘差縮放對齊後，模型被完整轉換為 8-bit 整數格式。

最終系統於 Xilinx VC707 FPGA 上完成實現，採用單引擎卷積模組與高效記憶體管理架構。實驗結果顯示系統在 RAF-DB 資料集上達成 86.31% 的準確率，並可即時推論每秒 73 幀，功耗僅為 0.803 瓦，驗證整數化量化 CNN 在邊緣設備中部署的可行性與實用性。

關鍵字：卷積神經網路(CNN)、整數量化、FPGA 實現、邊緣人工智慧

Abstract

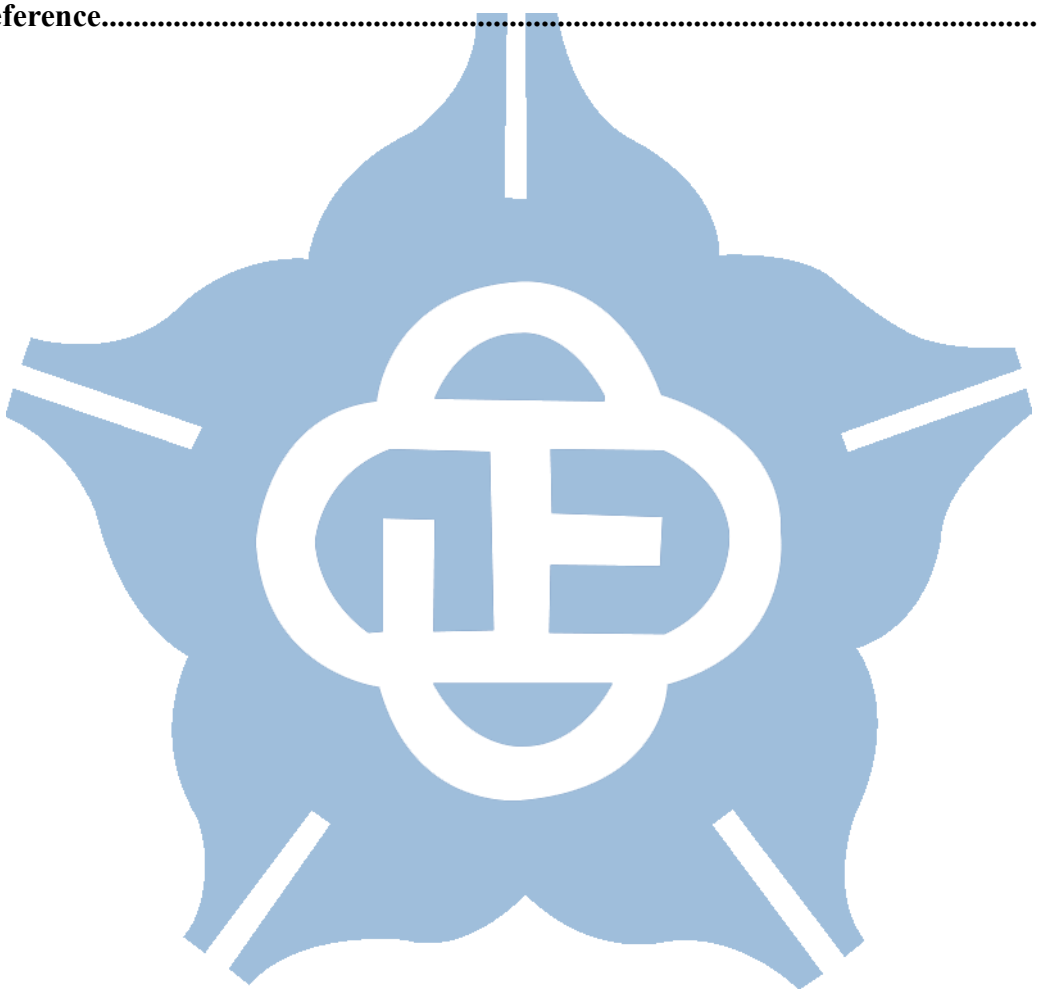
Facial expression recognition (FER) has become increasingly critical in various edge computing applications, such as healthcare, driver monitoring, and human-computer interaction. However, implementing deep learning-based FER models on resource-constrained platforms like FPGAs remains a major challenge due to their limited computational capacity and power budgets. This thesis presents a real-time FER system implemented on an FPGA using an integer-only quantized convolutional neural network (CNN) accelerator.

A customized and lightweight MobileNetV2 model was designed to process 100×100 resolution images, achieving a balance between accuracy and efficiency. The model was trained using a noise-resilient method called Erasing Attention Consistency (EAC) to improve performance on in-the-wild datasets. Quantization techniques, including per-channel symmetric weight quantization and residual scale alignment, were applied to fully convert the model into an 8-bit integer format.

A dedicated hardware accelerator was implemented on the Xilinx VC707 FPGA, featuring a unified convolution engine and efficient memory management. Experimental results demonstrate that the system achieves 86.31% accuracy on the RAF-DB dataset and reaches 73 frames per second (FPS), with low power consumption of only 0.803 W. This work demonstrates the feasibility of implementing high-performance FER systems on edge devices using quantized CNNs.

Keywords: convolutional neural network (CNN), integer quantization, FPGA implementation, edge AI.

Content



List of Figures

List of Tables

Chapter 1 Introduction

1.1 Introduction to Facial Expression Recognition and Its Applications

Before the advent of deep learning, human face analysis had already emerged as a significant topic in machine learning research. Researchers studied many aspects, including face detection, face alignment, face recognition, and facial expression recognition. In those early studies, the typical method for facial expression recognition was to apply handcraft filters to extract texture information from images and then process these features with a Support Vector Machine (SVM) [1], [2]. Even today, algorithmic methods continue to be used in certain research for processing facial images. For instance, local binary patterns serve as a fast and low-cost feature extraction method, making them appropriate for hardware implementation [3]. However, as image resolutions increase and full-color images become ubiquitous, traditional methods face new challenges, they are inadequate for modern high-resolution, full-color images. For example, local binary patterns (LBP) are designed for grayscale images, and using high-resolution images can greatly increase both the computational cost and the complexity of SVM-based classification.

In the past decade, the rapid development of deep learning has made it the mainstream approach for facial expression recognition. Its excellent feature extraction capabilities, combined with advances in hardware computing power, have enabled the processing of full-color, high-resolution images. One of the most commonly used deep learning architectures for image tasks is the convolutional neural network (CNN), which has been widely applied in facial expression recognition [4], [5]. Research in this area often compares different CNN architectures to determine their advantages in

expression recognition, whether in terms of inference speed or accuracy.

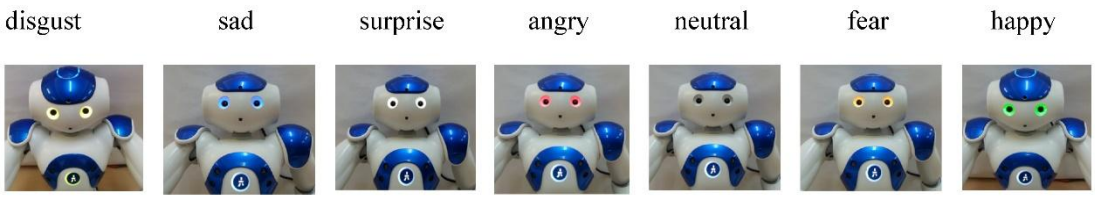
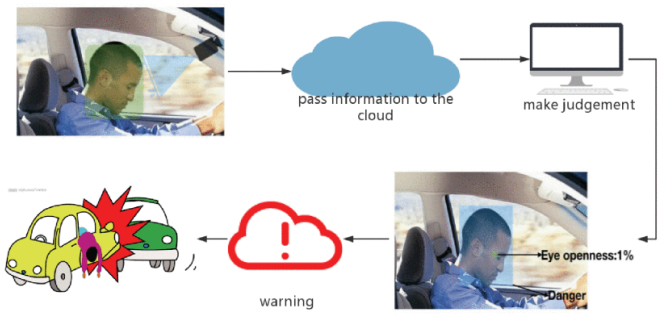


Figure 1.1 NAO color expression [4].

Facial expression recognition has been widely applied across many fields, as identifying human emotions is essential for improving quality of life. For example, in [4], facial expression recognition was deployed on NAO robots to develop social robots designed for companionship and service. As shown in Figure 1.1, the NAO robot demonstrates the capability to express seven distinct emotional states. If any emotional changes occur in the user, the robot promptly adjusts its behavior in real time by integrating the user's current emotional state with expert recommendations from psychologists and other professionals. It demonstrates an application in human-robot interaction. Furthermore, in [6], additional applications are demonstrated. For instance, in interrogations, the assistive system enables interrogators to detect a suspect's psychological state without requiring specialized training in psychological assessment. Another application involves monitoring drivers for signs of fatigue or anxiety, as these emotional states can increase the risk of accidents, necessitating timely alerts to the driver. Lastly, in the medical field, IoT-based facial expression recognition systems have been developed during the COVID-19 pandemic to monitor the psychological states of isolated patients. When negative emotions, such as anxiety or fear, are detected, the system alerts healthcare professionals, thereby supporting remote health monitoring [7].



(a) Cloud transmission process

(b) Edge transmission process

Figure 1.2 Differences in driver emotion analysis between cloud (a) and edge (b) [6].

From the aforementioned applications, it is evident that facial expression recognition is often integrated with IoT and deployed on edge devices. In particular, in the driver emotion detection application presented in [6], it is emphasized that edge computing must be utilized. Otherwise, cloud-based processing would delay the transmission of critical alerts to the driver, as shown in Figure 1.2, leaving insufficient time to prevent potential accidents. In other words, the models must be compact enough for edge devices and fast enough to quickly process data. Fast inference is critical for capturing the swift, often unconscious micro-expressions that occur in a fraction of a second. This capability also enables real-time detection and immediate feedback on the user's current emotional state. Moreover, accuracy is an equally important consideration, as efficient models can detect subtle changes in expression and extract fine-grained features, thereby supporting more comprehensive analysis and decision-making.

Generally, facial expression recognition tasks involve the following seven emotions: happy, surprised, sad, angry, disgusted, fear, and neutral. Through training on datasets, models can learn to distinguish these seven expressions. Commonly used

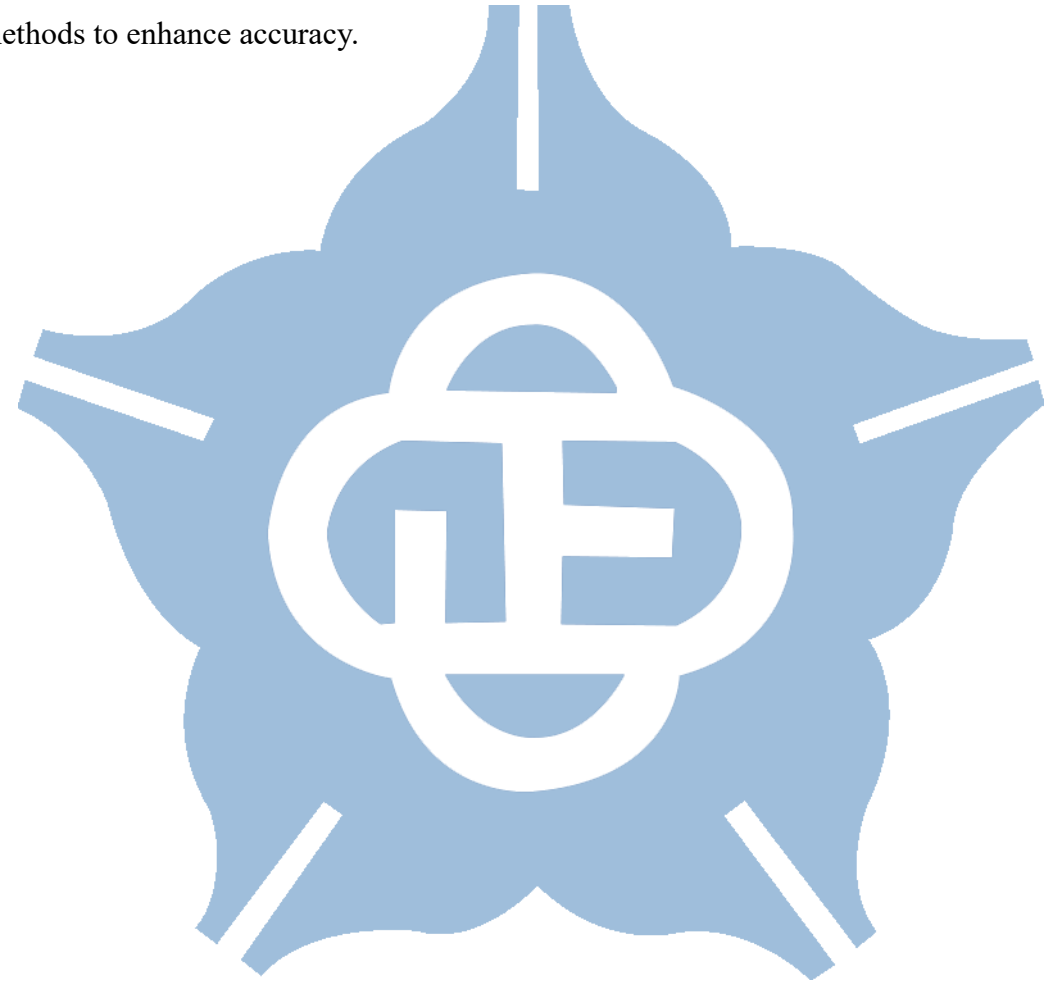
public datasets include Japanese Female Facial Expression (JAFFE) [8], FERPLUS [9], Real-world Affective Faces Database (RAF-DB) [10], and AffectNet [11]. Notably, FERPLUS and AffectNet include an additional emotion category for contempt.

The JAFFE dataset consists of 256×256 grayscale images collected in a laboratory setting featuring Japanese female expressions. Due to the relatively uniform subjects and conditions, the dataset often achieves testing accuracies as high as 90%. FERPLUS is based on the FER-2013 dataset, which comprises 64×64 grayscale images but contains many blurry or noisy labeled images. FERPLUS re-annotated the original images by employing a more strict voting mechanism to minimize subjective noisy labels, eliminated unclear images, and provided comprehensive label distribution information. Consequently, FERPLUS achieves a testing accuracy that is over 10% higher than that of FER-2013.

RAF-DB consists of full-color 100×100 images collected from the internet, encompassing a diverse range of genders, ages, and facial angles, with 12,271 training images and 3,068 testing images. This diversity enhances the generalizability of the applications beyond a single scenario. Similarly, AffectNet is another internet-collected dataset, larger in scale than RAF-DB with hundreds of thousands of images. However, due to variable image sizes and more frequent noisy labels, testing accuracies of less than 60% are common.

Despite the availability of these public datasets, several challenges remain, as they tend to contain noisy labels to some extent. It is primarily because facial expression recognition is inherently subjective, different individuals may interpret the same expression differently, and an expression can even simultaneously convey multiple emotions, known as compound expressions. However, facial expression recognition tasks require the clear identification of a single emotion, which presents a significant challenge. Another difficulty lies in the class imbalance within these datasets. While

laboratory-collected datasets might offer relatively balanced classes, those gathered from the internet typically contain a disproportionate number of "happy" and "neutral" images, with considerably fewer examples of "disgusted" and "fear." This imbalance results in insufficient training for these underrepresented classes, leading to lower testing accuracies. Consequently, facial expression recognition differs from conventional image classification tasks and may necessitate the use of additional methods to enhance accuracy.



1.2 Introduction to Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are deep learning architectures specifically designed for image-related tasks. Relying on fully-connected layers would result in enormous networks; for instance, the first layer alone would require a neuron for every pixel in the image, and such a high number of parameters could lead to overfitting. Instead, CNNs use convolution operations as shown in Figure 1.3, where filters scan across the image based on the concept of a receptive field. This approach significantly reduces both the number of parameters and the computational cost. The receptive field is based on the property of images where adjacent pixels are highly correlated, so filters collect information from a small, localized area to extract features. Larger-scale features can then be captured by stacking layers in a deep network, effectively expanding the filter's view. The VGG network [12] is built on this idea; it replaces large kernels with multiple small ones, reducing parameter count while also improving the network's ability to represent non-linear patterns thanks to the additional layers of activation functions such as ReLU.

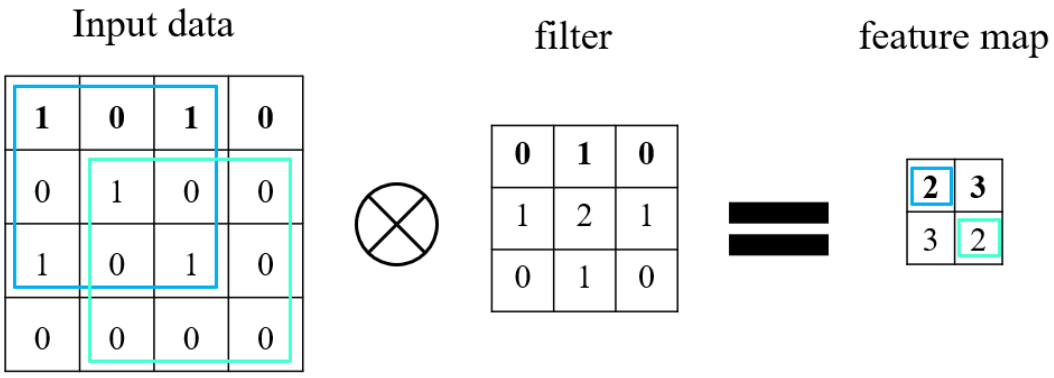


Figure 1.3 Example of the convolution operation.

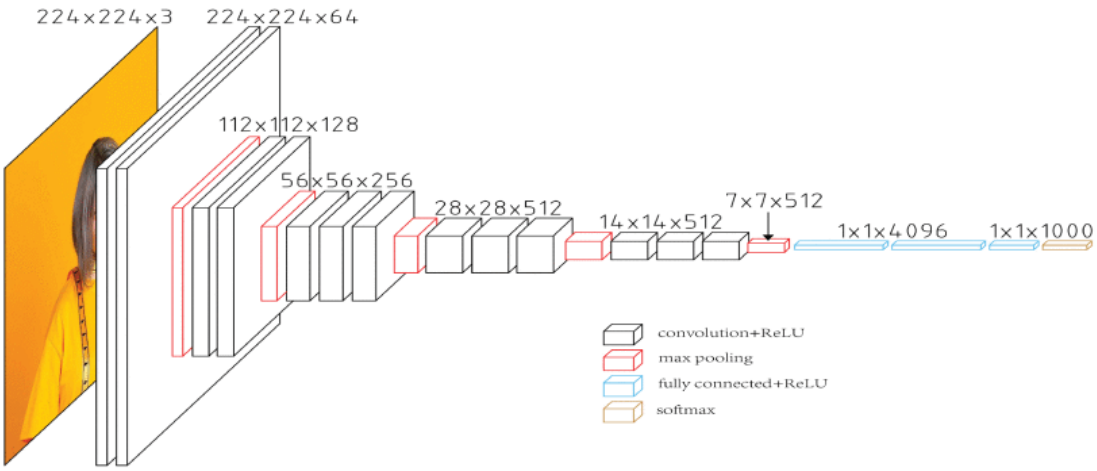


Figure 1.4 VGG16 architecture [5].

Figure 1.4 illustrates an image processed using VGG inference. The image progressively decreases in size through successive max pooling layers, and the convolution kernels used are of small size. Although VGG was considered a large network at the time, its deep and expansive structure enabled it to stand out in the 2014 ImageNet Challenge.

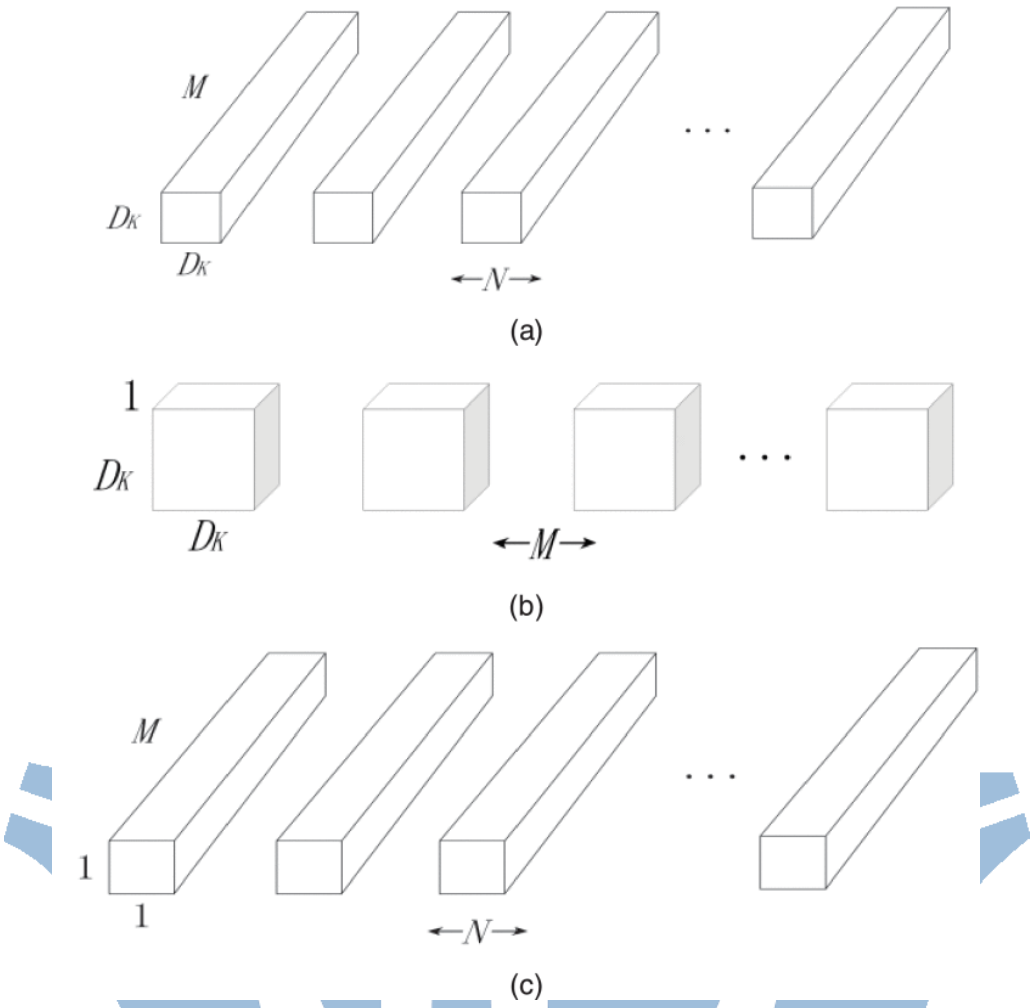


Figure 1.5 (a) Standard convolution. (b) Depthwise convolution. (c) Pointwise convolution [13].

VGG demonstrated that increasing network depth can improve model performance, laying the foundation for more advanced architectures. Subsequent research explored even deeper designs, but it was found that training very deep networks often led to inferior performance compared to shallower ones. The breakthrough came in 2015 with the introduction of ResNet, which revolutionized deep learning through the use of residual connections. These connections preserve information from previous blocks, preventing gradient decay caused by the chain-like structure of deep networks and effectively addressing the vanishing gradient problem. As a result, networks with up to 152 layers could be successfully trained. Later models,

such as DenseNet, further emphasized the benefits of deeper architectures. Today, many convolutional neural networks are inspired by ResNet, with numerous studies using it as a baseline for comparison. ResNet thus stands as a major milestone in the development of modern machine learning.

As networks have grown deeper, they have achieved excellent results in various tasks. However, the increased demands on computational resources have made it challenging to deploy these models on mobile devices or embedded systems, which has led to the exploration of lightweight model solutions. Even with the use of small filters in deep architectures, the cumulative cost of convolution operations remains substantial. To address the issue, MobileNet [13] introduced depthwise separable convolution in 2017, as illustrated in Figure 1.5, which decomposes standard convolution operations into two separate steps. Specifically, it comprises a depthwise convolution that performs convolution on each individual channel, and a pointwise convolution that uses a 1×1 kernel to combine information across different channels, thereby significantly reducing both computation and the number of parameters while maintaining functionality. Later, MobileNetV2 [14] further improved the accuracy of lightweight models by incorporating inverted residual blocks. Finally, MobileNetV3 [15] utilized Network Architecture Search (NAS) and the Squeeze-and-Excitation (SE) module to further condense the network size while preserving accuracy.

Section 1.1 noted that facial expression recognition tasks are often deployed on IoT and edge devices. As a result, numerous studies have adopted lightweight network architectures such as MobileNet. For example, in [16], an A-MobileNet was designed based on MobileNet, with additional techniques like Center Loss and the Convolutional Block Attention Module (CBAM) applied to enhance accuracy. In [17], a custom lightweight CNN model was proposed, it was specifically optimized for low-power and real-time environments, using simple convolutional and pooling layers along with

dropout regularization. In [18], ZenNet-SA was introduced as an efficient lightweight neural network that integrates a high-performance backbone automatically generated via ZenNAS with a feature distillation module using Shuffle Attention (SA). ZenNet-SA combines identity and attention-based branches to refine features while keeping computational complexity low.

These studies show that research in lightweight facial expression recognition typically starts with a lightweight network model and then applies optimization techniques to boost accuracy, rather than beginning with a large architecture and then reducing its size through methods like knowledge distillation or pruning. This insight guides the direction of the present study. However, if the objective is to achieve higher facial expression recognition accuracy, larger networks such as those in the VGG and ResNet series are necessary. Therefore, a trade-off between accuracy and complexity must be considered based on the specific application.

1.3 Facial Expression Recognition Model Training Methods

Section 1.1 discussed how in-the-wild facial expression recognition (FER) datasets often suffer from noisy labels and low-quality images. Recent studies have proposed various strategies to address these challenges.

Due to environmental variations, in-the-wild FER data typically exhibits intra-class discrepancy and inter-class similarity, which undermine the discriminative power of commonly used loss functions. For instance, the softmax loss struggles to create sufficiently separated feature representations across classes, while the center loss, which assigns equal importance to each feature dimension, may retain noise or features irrelevant to expression classification.

To overcome these issues, [19] introduced the Deep Attentive Center Loss (DACL), which integrates an attention mechanism into the center loss. Rather than treating all feature dimensions equally, DACL utilizes a Contextual Encoding (CE) Unit to extract an encoded latent feature vector and calculates the center loss by assigning attention weights that reflect the importance of each feature component.

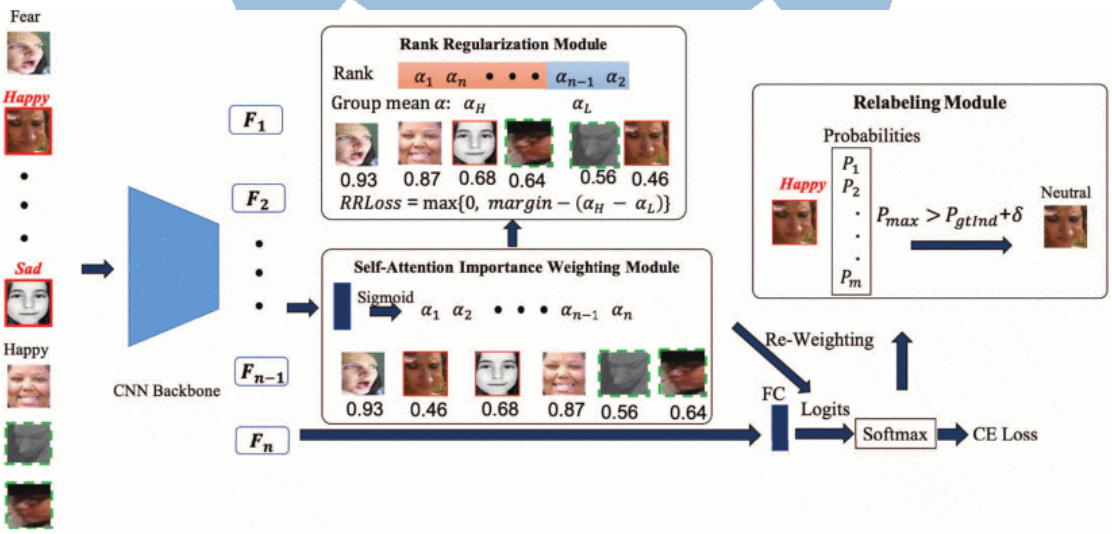


Figure 1.6 The pipeline of Self-Cure Network (SCN) [20].

In response to the challenge of noisy labels, [20] proposed the Self-Cure Network (SCN). SCN distinguishes between reliable samples and uncertain ones by assigning individualized weights to each instance, and then ranks these weights according to their magnitude, thereby filtering out noisy labels, as illustrated in Figure 1.6. Moreover, the relabeling mechanism can automatically modify incorrect annotations. However, since SCN's weight assignment is entirely based on the confidence scores of a pretrained model, it can still be prone to errors, especially in the early stages of training.

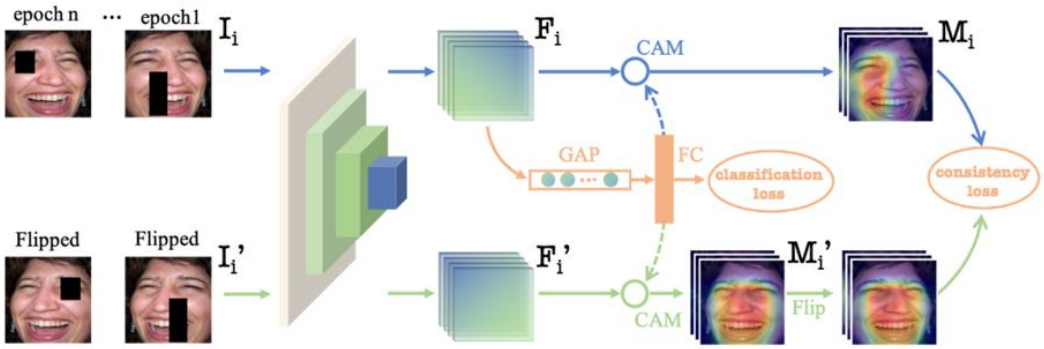


Figure 1.7 The framework of the Erasing Attention Consistency (EAC) [21].

Building on the above insight, [21] observed that SCN tends to encourage the model to focus on local features when distinguishing noisy labels. To address the limitation of excessive local focus and insufficient global understanding, the authors introduced Erasing Attention Consistency (EAC), which enforces global feature learning. EAC employs random erasing as a data augmentation technique to remove local features and introduces a consistency loss between the attention maps of original and horizontally flipped images. Consistency loss penalizes attention misalignment, thereby encouraging the model to focus on more holistic and meaningful regions. Unlike ensemble or multi-branch approaches, EAC uses Class Activation Mapping (CAM) to regularize attention alignment without altering base network structure, as illustrated in Figure 1.7.

While the methods described above primarily focus on noisy labels and feature learning, [22] tackled another major challenge in FER: class imbalance. An imbalanced dataset can hinder the effective learning of classes with few samples and reduce the model's generalization ability. To address the issue of insufficient representation and poor learning performance for underrepresented classes, the authors proposed Meta-Face2Exp, a framework that leverages unlabeled Face Recognition (FR) datasets to enhance expression recognition. Meta-Face2Exp consists of a base network and an adaptation network. Both networks share the same structure but maintain independent model weights. The base network is trained on balanced FER data and generates pseudo-labels for unlabeled FR data. The adaptation network learns from these pseudo-labeled FR samples. Feedback from the adaptation network is then used to improve the base network in a circuit feedback loop, enhancing its ability to generalize to underrepresented expressions. Through the dual-network strategy, Meta-Face2Exp effectively increases the diversity of training data and helps to balance the class distribution in FER.

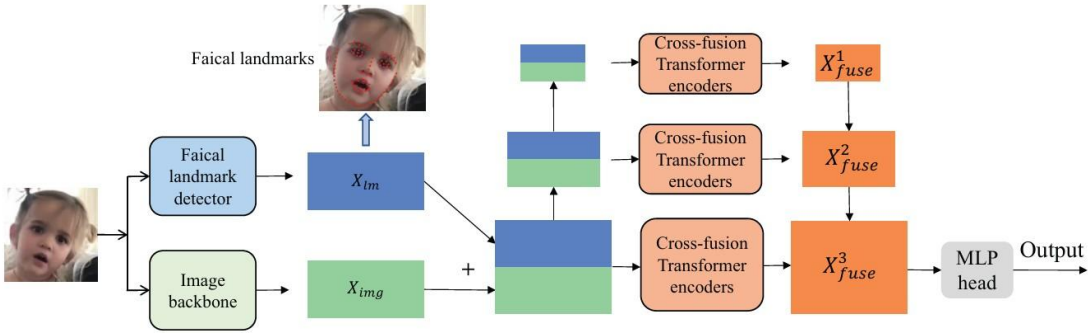


Figure 1.8 The architecture of POSTER [23].

Finally, [23] introduced a powerful dual-stream architecture known as the Pyramid crOssfuSion TransformER (POSTER), as shown in Figure 1.8. POSTER is designed to address intra-class discrepancy and inter-class similarity. It employs two parallel streams: the image stream extracts global appearance features, while the landmark

stream captures local information from facial landmarks. The two streams interact through a cross-fusion transformer mechanism. In the process, queries from the image stream attend to the keys and values from the landmark stream to emphasize salient regions. Simultaneously, the landmark stream's queries attend to the keys and values from the image stream, supplementing its lack of relative positional information. The bidirectional attention allows the network to simultaneously emphasize local facial regions and integrate global appearance. POSTER achieves state-of-the-art results (e.g., 92.05% accuracy on RAF-DB); however, its large-scale architecture and high computational resource requirements make it challenging to implement on edge devices.

To implement on edge devices such as FPGAs, lightweight and modular architectures are preferred. The backbones used in [19], [20], and [21] are all based on ResNet-18 and require no additional network branches or large-scale augmentation, making them ideal for direct application in lightweight model implementations. However, it is important to note that these methods heavily rely on pre-training to ensure that the initial model is suitable for fine-tuning. Consequently, the quality of the pre-training significantly influences final performance.

1.4 Quantization Methods in Neural Networks

With the rapid success of deep learning models in fields such as computer vision, speech recognition, and natural language processing, the number of parameters and the computational complexity of these networks have increased significantly. The increase in model size and computational demands poses challenges for deploying them on resource-constrained embedded or edge devices. Maintaining high performance with limited hardware resources has become a major challenge. Quantization, which effectively reduces memory usage and accelerates computations, has become one of the key research directions for building efficient neural networks.

In 2016, Courbariaux et al. introduced the Binarized Neural Network (BNN) [24], which was the first to employ quantization using only binary values, +1 and -1. BNN demonstrated that even extremely compressed networks can still perform reasonably well. In addition, because the computations are done with binary values, multiplications and additions can be replaced by simpler operations such as bit shifting or look-up tables (LUTs), enabling highly cost-effective hardware implementations.

Around the same time, Zhou et al. proposed DoReFa-Net [25] for convolutional neural networks (CNNs), which quantizes weights, activations, and gradients into fixed-point numbers using a method called Quantization Aware Training (QAT). The network was successfully quantized to 8-bit while maintaining higher accuracy than Binarized Neural Networks (BNNs). To support backpropagation through non-differentiable quantization operations, DoReFa-Net employs the Straight-Through Estimator (STE), allowing flexibility across various bit widths. The design has since inspired many subsequent works in the field.

In 2018, Google presented an Integer-Arithmetic-Only (IAO) framework [26]. Using uniform quantization with scale factors and zero points as illustrated in Equation

1.1, where real numbers r_a (a = 1, 2 or 3) are mapped to fixed-point representations q_a using scale factors S_a and zero points Z_a, the framework maps real-valued data precisely into an integer domain. This work allows all inference operations, including convolutions, bias additions, and fused operators, to be performed entirely with integers. Equation 1.2 represents the transformation from real-number multiplication to integer multiplication, while Equation 1.3 derives M by rearranging the individual scale factors S_a. The multiplier M can be decomposed into a power-of-two component and a fractional part M_0, as shown in Equation 1.4, which allows it to be efficiently implemented using bit shifting and fixed-point multiplication. Floating-point operations are only required for quantization and dequantization, with all other operations performed in the integer domain. Therefore, it is well-suited for deployment on hardware platforms such as ARM NEON, DSPs, and FPGAs. Experimental results on large-scale datasets like ImageNet show that the method loses only about 1%–2% in accuracy from original float32 model, indicating a good balance between performance and efficiency.

$$r_a = S_a(q_a - Z_a) \tag{1.1}$$

$$S_3(q_3 - Z_3) = S_1(q_1 - Z_1)S_2(q_2 - Z_2) \tag{1.2}$$

$$q_3 = Z_3 + M(q_1 - Z_1)(q_2 - Z_2) \tag{1.3}$$

$$M := 2^{-k}M_0 \tag{1.4}$$

Integer quantization involves several key parameters. One example is whether the zero point is set to 0, which determines the use of symmetric or asymmetric quantization. As explained in [27], symmetric quantization fixes the zero point at 0, simplifying both the quantization and dequantization processes. This method is particularly suitable for low-power hardware such as DSPs. On the other hand, asymmetric quantization can

more accurately account for data shifts and is commonly used for activation quantization to enhance model accuracy, although accommodating the shift introduces additional hardware overhead. Another important factor is the granularity of quantization, which can be applied either per channel or per layer. Per-channel quantization assigns a distinct scale to each channel, generally preserving model accuracy. In contrast, per-layer quantization uses a single scale across all channels, potentially leading to a noticeable drop in accuracy due to the disregard of inter-channel differences.

In view of these considerations, the paper recommends adopting per-channel quantization for weights and per-layer quantization for activations. It allows the quantization process to better handle the significant variation across weight channels caused by operations such as batch normalization, while maintaining efficiency and sufficient precision for activations, whose dynamic ranges are often narrower and more consistent. The study also indicates that using asymmetric quantization for activations leads to slightly better accuracy because it can better adapt to shifted data distributions. On the other hand, symmetric quantization is more efficient and effective for weights when applied on a per-channel basis. This combination achieves a strong balance between implementation simplicity and inference accuracy, making it a robust baseline for post-training quantization.

$$y = PACT(x) = 0.5(|x| - |x - \alpha| + \alpha) = \begin{cases} 0, x \in (-\infty, 0) \\ x, x \in [0, \alpha) \\ \alpha, x \in [\alpha, +\infty] \end{cases} \quad (1.5)$$

More advanced quantization techniques, such as PACT [28], focus on optimizing the dynamic range of activations. Unlike traditional methods that primarily concentrate on weight quantization, activations actually consume a larger share of resources during inference. However, direct quantization of activations often leads to substantial

accuracy degradation. To address the challenge of balancing aggressive compression with minimal loss of representational accuracy, PACT introduces a trainable parameter α, to replace the traditional ReLU activation function and uses it to control the clipping threshold of activations (see Equation 1.5). The PACT method helps the network maintain both expressive capacity and task performance under quantization, even at low bit widths (e.g., 4-bit or lower). The method employs a Straight-Through Estimator (STE) to backpropagate gradients for α, making it compatible with DoReFa-Net for full-model low-bit fixed-point quantization.

$$\frac{\partial \hat{v}}{\partial s} = \begin{cases} -v/s + [v/s] & if\ -Q_N < v/s < Q_P \\ -Q_N & if\ v/s \leq -Q_N \\ Q_P & if\ v/s \geq Q_P \end{cases} \quad (1.6)$$

Finally, LSQ [29] proposes a "Learned Step Size" approach to enhance the inference performance of low-bit models. Unlike prior methods that rely on fixed step sizes or quantization scales determined from statistical distributions, approaches that often fail to account for layer-specific variability, LSQ treats the step size as a trainable parameter. The parameter is optimized during backpropagation (see Equation 1.6), enabling dynamic adjustment of the quantization scale for each layer to minimize task loss and thereby improve overall accuracy. LSQ also uses the STE to handle the non-differentiable rounding operation. Additionally, the LSQ method can be combined with integer quantization to ensure high compatibility with hardware deployment. Experimental results show that LSQ achieves better accuracy than PACT under similar constraints.

1.5 Related Work of Facial Expression Recognition on FPGA

With the rapid advancement of deep learning algorithms, numerous studies have attempted to implement facial expression recognition (FER) systems on FPGA platforms, aiming to combine the advantages of low power consumption with real-time processing.

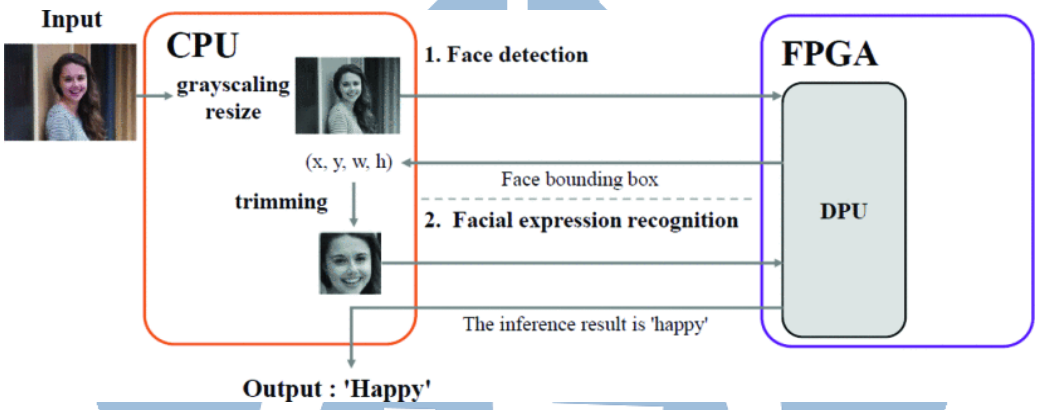


Figure 1.9 System configuration [30].

Reference [30] proposed a facial expression recognition system implemented on the Xilinx KV260 FPGA. The system utilizes DenseBox for face detection and a convolutional neural network (CNN) for expression recognition. To enhance overall performance, multi-threading was incorporated to increase the utilization of the deep processing unit (DPU), achieving real-time processing at 25 frames per second (FPS) (see Figure 1.9). However, the system achieved only 67.4% accuracy on the FER-2013 dataset, which may limit its robustness for practical implementation. Additionally, the processing speeds between the CPU and DPU are balanced only when using a 48×48 image resolution, which may strain the DPU's processing capability under larger and more complex input settings. Even though multi-threaded CPU processing is employed to prevent underutilization of the DPU, the DPU itself still poses a limitation in

processing more complex models or higher-resolution images, making it a potential bottleneck as system demands scale.

In [3], a FER system that combines Local Binary Patterns (LBP) and Histogram of Oriented Gradients (HoG) was implemented on the Xilinx Zynq 7020 SoC. The system applies the Viola-Jones algorithm for face detection. By leveraging traditional image processing techniques to extract features, it uses a simple artificial neural network (ANN) for expression classification, resulting in a lightweight and computationally efficient design well-suited for embedded systems. However, since it does not utilize modern deep learning models, its recognition performance is inferior to that of current mainstream CNN architectures.

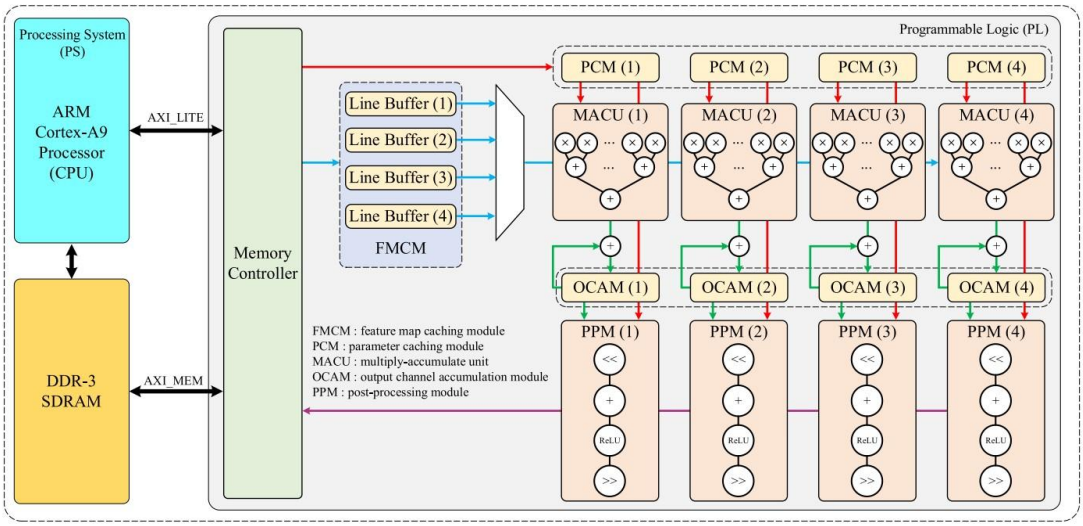


Figure 1.10 Hardware architecture of the CNN accelerator [31].

Ref. [31] presented a lightweight CNN architecture designed for integer computations on the Xilinx ZC706 SoC. To enhance model accuracy, a custom FERPlus-A training dataset was constructed, resulting in 86.58% accuracy on the FERPlus test set. The system employs Log-Level Threshold Quantization (LLTQ) for near-lossless precision quantization and uses a parallel processing strategy in its hardware design. Specifically, it processes four channels simultaneously for each 3×3 convolution, as shown in Figure 1.10. The design supports real-time facial expression

recognition at 10 frames per second. Although it achieves a favorable balance between accuracy and computational efficiency, its generalization capability on more diverse datasets, such as RAF-DB and AffectNet, remains to be validated. The limitation arises from the FERPlus training set, which was derived from FER2013 and lacks sufficient variability in lighting conditions, head poses, and demographic diversity.

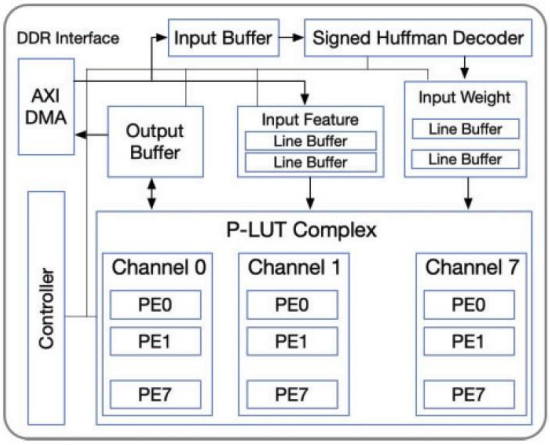
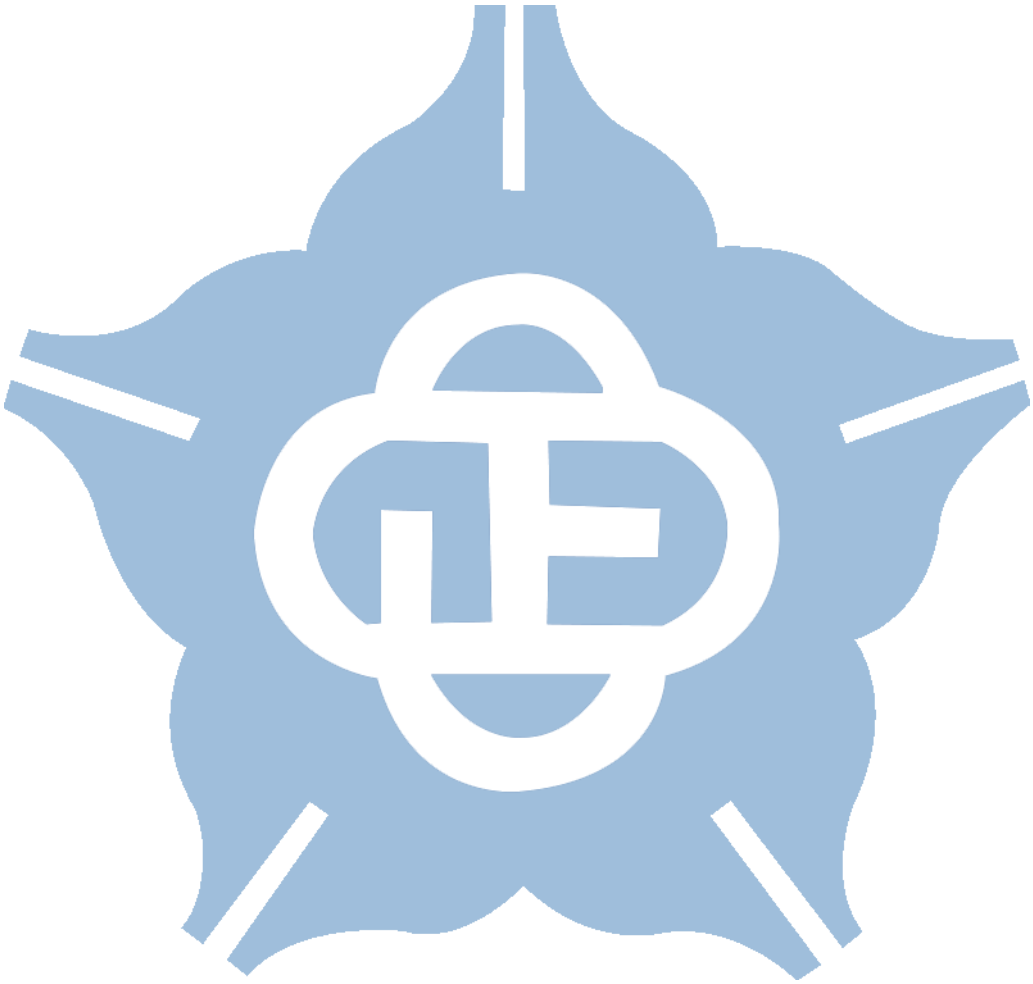


Figure 1.11 The P-LUT accelerator architecture [32].

In [32], a low-power FPGA architecture designed for edge computing was introduced. The design integrates the proposed Integer-Only Scalar Power-of-Two (IOS-PoT) quantization with a P-LUT computation method to create an efficient deep learning inference platform. The system, implemented on the Xilinx KV260 FPGA, first trains a model with SCN method on the RAF-DB dataset. Then, IOS-PoT quantization is applied to learn appropriate scaling factors, replacing conventional multiplications with efficient bit-shift operations based on power-of-two representations. Both weights and activations are quantized to 3/4-bit, which significantly reduces computational and memory demands. Hardware optimization is further achieved by leveraging P-LUT, which directly maps power-of-two quantized values to pre-computed results in a lookup table, and by implementing 8-channel parallel processing to accelerate computation. As shown in Figure 1.11, the overall

accelerator architecture achieves twice the resource efficiency, reduces the memory footprint by a factor of 1.45, and offers three times the computation efficiency. Nonetheless, the model's accuracy decreases to approximately 77% after quantization, indicating that concerns regarding accuracy loss remain for practical applications, thus limiting its overall applicability.



1.6 MobileNetV2 Hardware Accelerator Architecture

As the demand for CNN deployment on edge platforms continues to grow, specialized hardware accelerators must satisfy strict constraints on power, latency, and resource usage. General-purpose accelerators support standard CNN models but often struggle with lightweight architectures like MobileNetV2, which uses depthwise separable convolutions and inverted residual blocks to reduce parameters and computation. These features introduce challenges such as irregular memory access and the need for efficient pointwise convolution handling.

Recently, full-integer quantization has become an increasingly popular technique for accelerating CNN models on edge devices, as it reduces both memory and computation requirements while maintaining acceptable accuracy. Building on this trend, Weixiong Jiang et al. proposed a high-throughput, full-dataflow MobileNetV2 accelerator specifically designed for FPGA platforms [33]. The work introduced an optimized architecture that fully exploits the parallelism of depthwise separable convolutions, minimizes memory bottlenecks, and carefully balances the trade-offs between hardware utilization and model accuracy. By integrating integer-quantization optimizations and tailoring the dataflow design to the unique characteristics of MobileNetV2, their accelerator achieved significant improvements in inference speed and energy efficiency compared to prior CNN accelerators.

In addition, the design tackles the challenge of scale mismatch in residual connections during full-integer quantization by applying a scaling factor alignment strategy. The approach simplifies hardware implementation by ensuring that the scales of operands in addition are consistent. During quantization, the scale factors are aligned so that the addition operation in the residual path can be performed directly in the

integer domain without requiring extra dequantization. It reduces hardware complexity, enabling the efficient implementation of residual connections through straightforward integer addition, while maintaining nearly lossless accuracy.

In contrast to the aforementioned work, another FPGA-based MobileNetV2 accelerator was proposed by Shun Yan et al. [34], with a design strategy that emphasizes architectural optimization based on the structural characteristics of MobileNetV2. The accelerator employs a single convolution engine capable of efficiently handling both pointwise and depthwise separable convolutions, thereby improving hardware reusability and simplifying implementation.

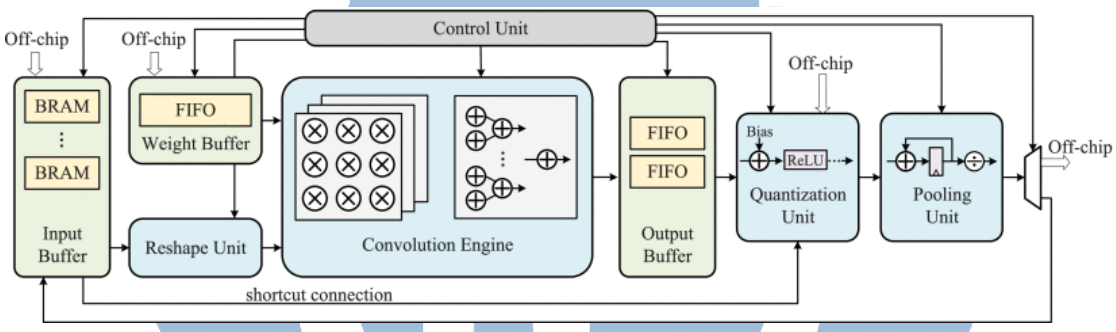


Figure 1.12 Design overview of the MobileNetV2 accelerator [34].

To further enhance performance, the authors introduce three hardware-level optimizations, as illustrated in the architecture diagrams of the paper. The overall system design is presented in Figure 1.12, which provides a comprehensive overview of the accelerator's architecture, including data paths, memory buffers, and control flow. The design adopts a single convolution engine that supports various convolution types and integrates pipeline mechanisms to overlap computation and data movement, thereby maximizing resource utilization and throughput.

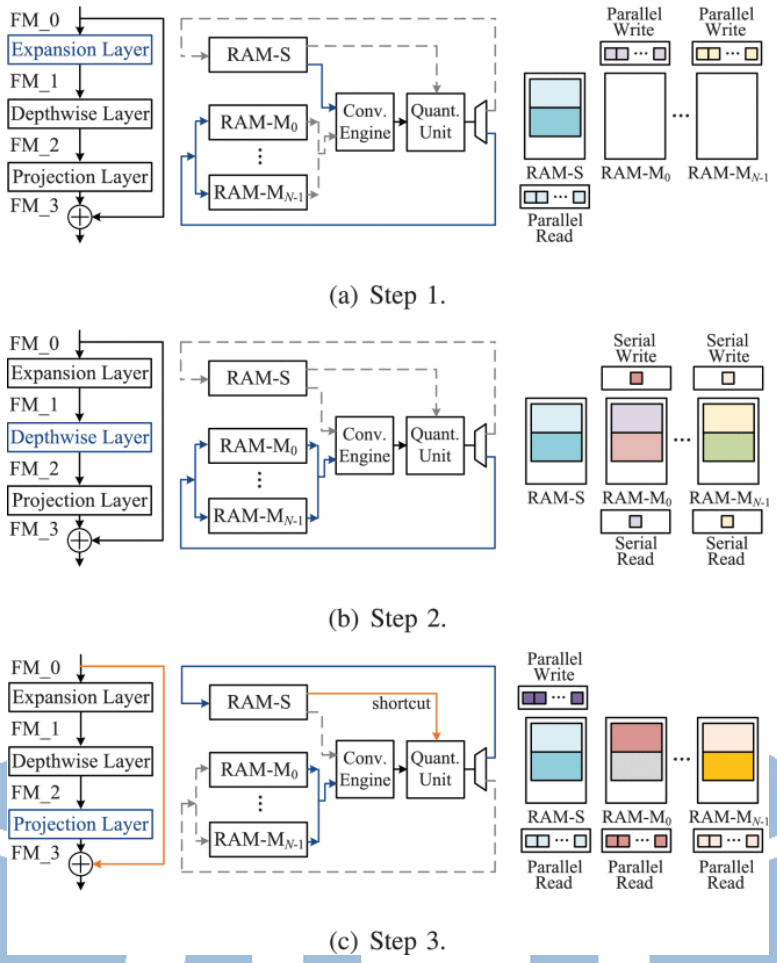
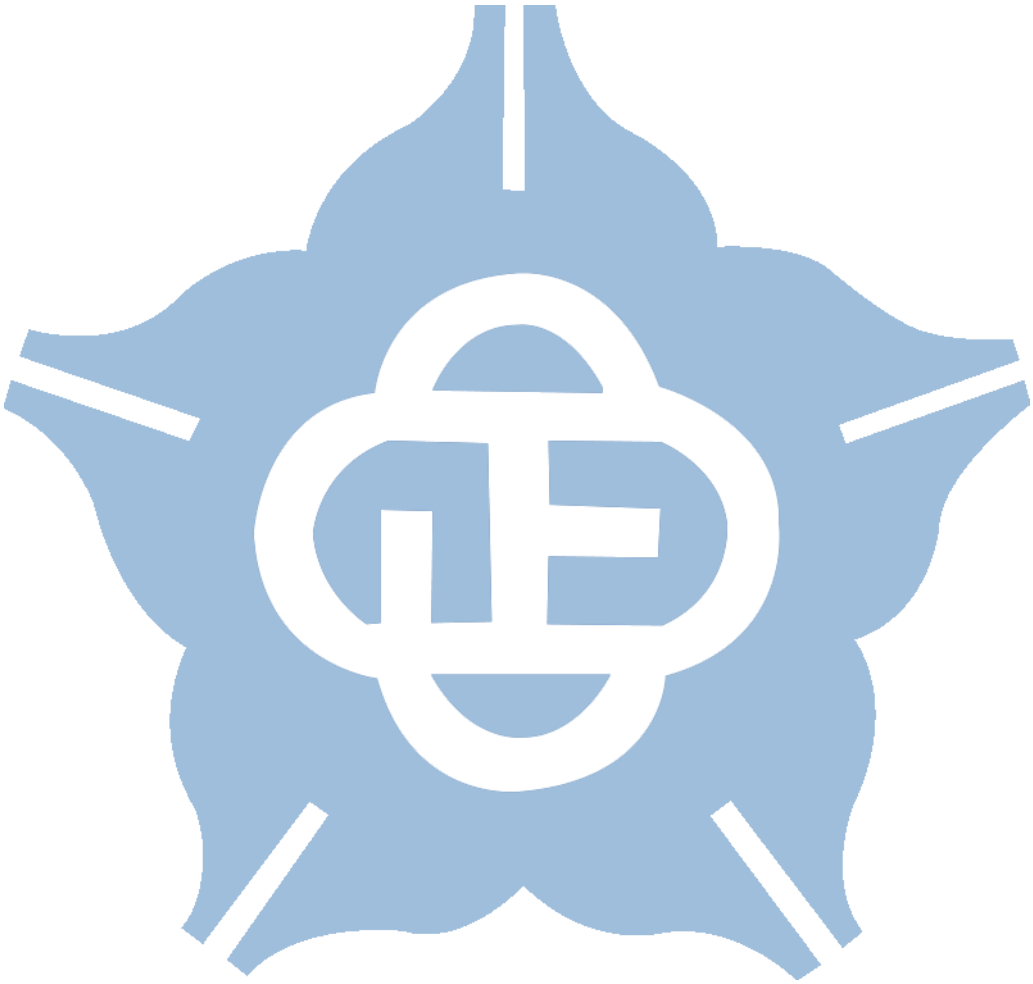


Figure 1.13 Architecture and operation flow for the bottleneck block and shortcut connection [34].

Building upon this foundation, a customized architecture for the bottleneck block is introduced to reduce data transmission overhead by reusing intermediate feature maps within the on-chip memory. As shown in Figure 1.13, the operation flow is divided into three steps corresponding to the expansion, depthwise, and projection layers of the bottleneck structure. Figure 1.13 also illustrates how shortcut connections are efficiently handled using a ping-pong buffer scheme, which minimizes off-chip memory access and improves processing continuity. In addition, an operation-separate padding scheme is employed to eliminate invalid data windows during convolution, and a configurable adder tree is designed to accommodate different convolution types

using a unified computation structure, thereby reducing hardware overhead while preserving system performance.



1.7 Motivation

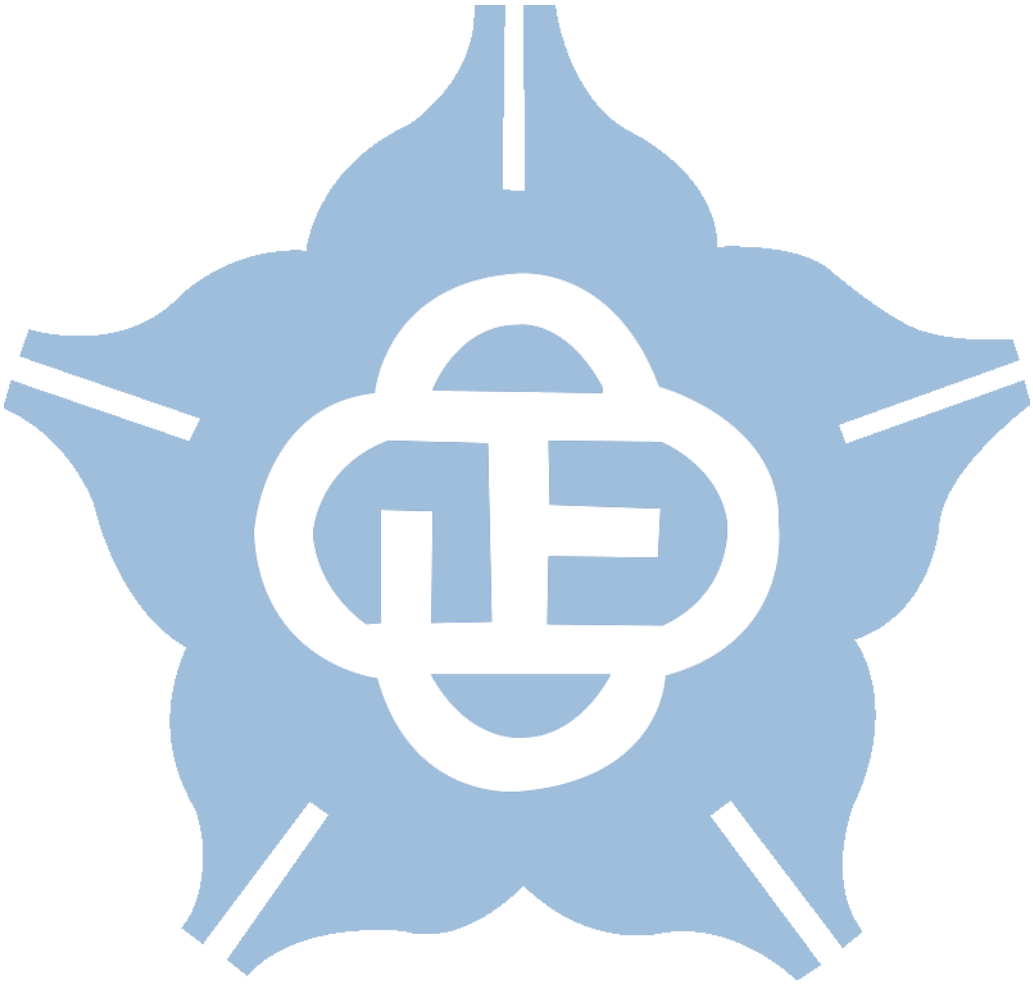
Recent applications of FER have been increasingly integrated into IoT environments or deployed on edge devices. These systems are required to support real-time inference with lightweight architectures while maintaining adequate accuracy under practical constraints. Meeting these objectives is non-trivial, as it requires a holistic workflow that bridges both software and hardware components to ensure a balanced and efficient system.

In this thesis, FER is utilized within a hospital triage classification robot that continuously monitors patients' conditions in real time by analyzing their facial expressions. To address the resource limitations, a modified MobileNetV2 is adopted as the backbone lightweight model, optimized for low-resolution inputs (100×100 pixels). In addition, label noise mitigation techniques are incorporated to improve model robustness and classification performance. To further enhance implementation efficiency, the model is quantized to 8-bit integer precision, thereby reducing computational overhead while preserving accuracy. For real-time inference, the system is implemented on the Xilinx VC707 FPGA, with a custom hardware accelerator developed to meet stringent timing and throughput requirements.

The contributions of this work are summarized as follows:

1. A computationally efficient FER model is developed, achieving 86% accuracy on the RAF-DB dataset. The model significantly reduces parameters and computational complexity, making it well-suited for real-world edge applications.
2. Integer quantization techniques are applied to convert both weights and activations to an 8-bit representation. It alleviates computational bottlenecks and enables efficient inference on resource-constrained platforms.
3. A dedicated hardware accelerator is implemented on a Xilinx FPGA, allowing the

system to meet real-time processing requirements and ensuring reliable implementation in clinical triage environments.



1.8 Thesis Organization

This thesis presents the complete workflow from the design of a facial expression recognition model to its implementation on an FPGA. The structure of the thesis is organized as follows:

Chapter 1 provides an overview of FER applications and discusses the training methods used for in the wild FER datasets. It also introduces commonly used quantization techniques when implementing convolutional neural networks (CNNs) to FPGAs, as well as the design of hardware accelerator architectures.

Chapter 2 introduces the use of MobileNetV2 as the backbone model. It describes how the architecture was modified to create a more lightweight version, and how quantization techniques were applied to prepare the model for hardware implementation.

Chapter 3 focuses on the hardware implementation of the CNN, particularly for MobileNetV2. It explains how the architecture was adapted to design a hardware accelerator optimized for the model.

Finally, Chapter 4 presents the conclusions of this study and outlines possible directions for future work.

Chapter 2 Software Implementation

2.1 Evaluation of Benchmark Models under Erasing Attention Consistency

Implementing FER systems on FPGA platforms demands models that balance high accuracy with computational efficiency to meet the limited hardware resource and strict latency requirements. Accordingly, this section introduces an end-to-end software workflow for preparing a lightweight facial expression recognition model for FPGA implementation.

As outlined in Section 1.3, several recent methods address noisy labels in FER datasets using ResNet-based backbones. These approaches are highly adaptable and can be generalized to other architectures. After comparing multiple candidate models with varying complexity, the one offering the best trade-off between accuracy and computational cost was selected.

This work employs the training approach proposed in [21], specifically the Erasing Attention Consistency (EAC) framework. EAC demonstrates strong performance in addressing noisy labels. A key advantage of EAC is its ability to regularize attention alignment through a consistency loss function, ensuring consistent attention maps between original and flipped images. The consistency guidance is provided by CAM, which identifies discriminative regions within images by projecting output weights onto the final feature maps. Notably, this method does not necessitate additional branches or modifications to the underlying network architecture. Consequently, the model can be quantized directly after training without requiring further adjustments.

To benchmark model performance, each candidate model was trained using a consistent pipeline. Pre-training was conducted on the large-scale C-MS-Celeb face

recognition dataset [35]. During training, input images from the RAF-DB dataset were maintained at a resolution of 100×100 pixels. The training process had 80 epochs with a batch size of 64, using a learning rate of 0.001. The Adam optimizer was employed with a weight decay of 0.0001. Data augmentation included horizontal flipping and random erasing, following the EAC methodology.

After pre-training, the benchmark models are fine-tuned on the RAF-DB dataset using both transfer learning and EAC training strategies. The "Baseline" column in Table 2.1 refers to the results obtained from direct training, without the use of pretraining or any auxiliary methods, and serves as the reference model for comparison. The “Transfer” column presents the performance achieved by applying transfer learning, in which the models are first pre-trained on the large-scale C-MS-Celeb dataset and then fine-tuned on RAF-DB. Transfer learning leverages the rich feature representations learned from the source dataset to improve performance on the smaller target dataset. The “EAC” column, on the other hand, represents results achieved after further applying the EAC method on top of the transfer learning setup. A comparative analysis of the training performance using both methods is presented in Table 2.1. The results show that EAC substantially improves model accuracy while preserving a lightweight structure that is well-suited for efficient FPGA implementation.

Table 2.1 Comparison of model performance under different training methods.

Model	**Baseline Acc. (%)**	**Transfer Acc. (%)**	**EAC Acc. (%)**	**Parameters**	**MACs**
VGG13	82.99	85.14	85.56	9.41M	2.69G
MobileNetV1	77.80	84.81	87.81	3.21M	143M
MobileNetV2	78.68	83.74	87.29	2.23M	81M
MobileNetV3small	75.29	82.89	84.26	1.67M	16M

Several key observations can be drawn from Table 2.1. First, VGG13 achieves a higher baseline accuracy compared to the MobileNet series, reflecting its stronger inherent capacity. However, the improvements gained by applying transfer learning and the EAC method to VGG13 are relatively minor, suggesting that most of the performance gains in this case are largely attributable to the benefits of transfer learning alone.

In contrast, the MobileNet models show clear and consistent improvements across the stages of baseline training, transfer learning, and EAC. This observation suggests that lightweight architectures, due to their limited capacity, benefit significantly from the additional regularization provided by the EAC framework, likely because their reduced capacity makes them more reliant on effective loss regularization to achieve meaningful feature learning.

Based on these findings, conducting a detailed comparative analysis among different MobileNet variants is essential for identifying the optimal architecture for implementation on edge devices. Thus, a thorough evaluation was performed on the trade-offs between accuracy, model size, and computational efficiency. While MobileNetV2 does not exhibit the highest absolute accuracy, it strikes a favorable balance between accuracy and efficiency. Compared to MobileNetV1, MobileNetV2 significantly reduces parameter count and computational complexity while achieving similar accuracy. Although MobileNetV3-small features fewer parameters, its lower accuracy and increased architecture complexity due to the inclusion of SE blocks and the Hard-Swish activation function restrict practical implementation. Consequently, MobileNetV2 was chosen due to its ability to sustain high recognition performance while complying with real-time system constraints and resource limitations, rendering it the most practical choice for FPGA implementation.

2.2 Architecture Refinement of MobileNetV2 Based on Workflow

After selecting MobileNetV2 as the backbone network, it is necessary to further optimize its architecture. The original MobileNetV2 was designed for the ImageNet dataset, which contains input images sized 224×224 pixels. However, the RAF-DB dataset used in this study consists of smaller images with dimensions of 100×100 pixels. To meet the requirements for efficient inference on these smaller images and to reduce computational costs on hardware with constrained resources, appropriate scaling of the original architecture must be implemented to accommodate the reduced input dimensions.

Therefore, to ensure that the model retains sufficient performance despite the architectural downsizing, a structured workflow was designed to guide the optimization process and maintain expected performance levels. Figure 2.1 illustrates the training workflow. The process begins by defining a modified MobileNetV2 architecture, which is initially trained directly on the RAF-DB dataset as a baseline evaluation. A baseline accuracy of at least 78% is required to avoid significant performance degradation compared to the original architecture, the workflow proceeds to the pre-training stage, followed by the application of the EAC method for fine-tuning.

If the fine-tuned model achieves an accuracy above 86%, it advances to the quantization stage. Otherwise, if the fine-tuned accuracy falls below 86%, the process returns to the initial step for further architectural modification. After quantization, the model's accuracy is evaluated, and hardware implementation proceeds only if its accuracy degradation is smaller than 1%. Although post-training quantization is fast and efficient, it lacks the ability to recover performance losses caused by quantization errors through fine-tuning. As a result, emphasis is placed on early architectural

adjustments to ensure robustness prior to quantization. The iterative process repeats until the model consistently satisfies the target accuracy requirement.

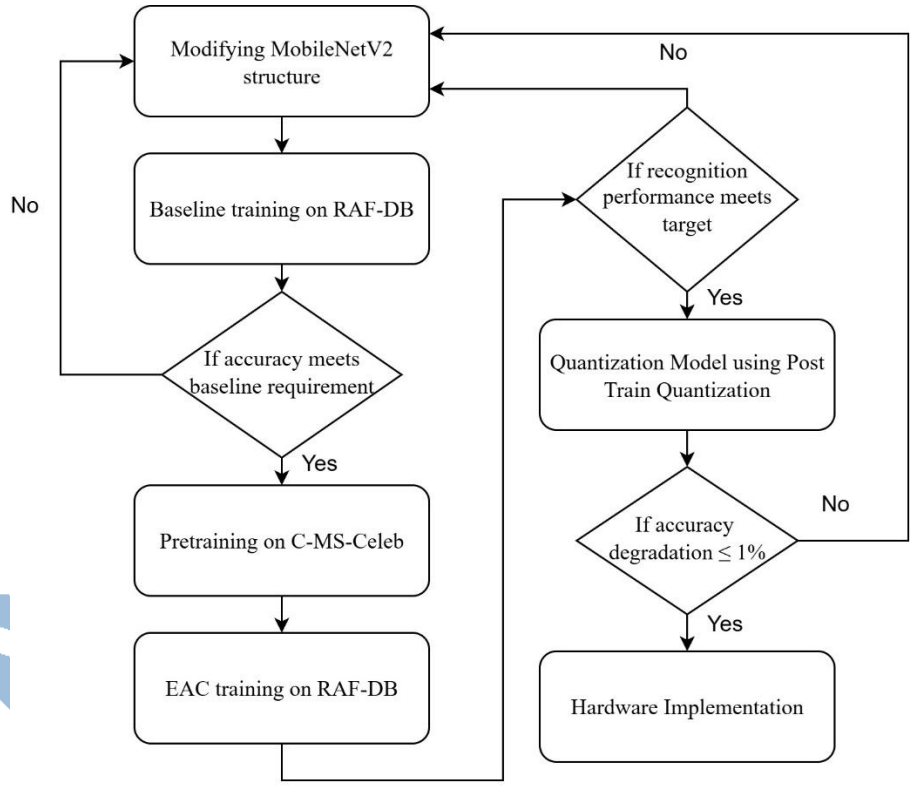


Figure 2.1 A training workflow for model architecture modification.

Based on empirical findings from multiple experiments, once a complete training cycle has been conducted using the original architecture, the observed performance difference between the baseline model and the EAC model can be used to estimate the potential effectiveness of EAC after future architectural modifications. This predictive approach facilitates early identification of promising architecture configurations during the development process, enabling subsequent fine-tuning based on actual training results, thus enhancing overall development efficiency and final model performance.

Table 2.2 summarizes the progressive refinement of the MobileNetV2 architecture throughout the training workflow. Each version represents a targeted modification designed to optimize model performance under hardware constraints. The initial version (V0) follows the original ImageNet configuration, while V1 reduces the

expansion factor to lower computation. V2 adjusts the stride setting to enlarge feature maps, as shown in Table 2.2, the architectural change results in a significant improvement in baseline accuracy compared to V1, indicating that larger feature maps contribute positively to the model's ability to capture expressive features. In theory, larger feature maps also improve spatial resolution and strengthen attention mechanisms like CAM. The final version applies additional reductions in channels and bottlenecks to produce a highly compact architecture suitable for edge device. These incremental modifications collectively enable the model to preserve high accuracy while significantly reducing computational complexity, thus demonstrating the effectiveness of the proposed design strategy. Nevertheless, the current results still require the subsequent application of the previously mentioned EAC method to further enhance training accuracy and improve the model's robustness against noisy labels.

Table 2.2 Progressive modification of MobileNetV2 architecture on baseline training.

Version	**Modification Description**	**Parameters**	**MACs**	**Acc. (%)**
V0	Original MobileNetV2 architecture designed for 224×224 input	2.23M	81M	78.68
V1	Reduced expansion factor from 6 to 4 across all bottleneck blocks	1.63M	58M	77.15
V2	Reduced the number of stride 2 blocks to enlarge feature map sizes for better accuracy	1.63M	156M	78.49
Final	Customized lightweight version with fewer bottlenecks and channels, optimized for 100×100 input	659K	68M	78.36

Table 2.3 presents the final architecture configuration of the optimized MobileNetV2 model. Each row corresponds to a specific layer or block in the network, detailing its structure and function.

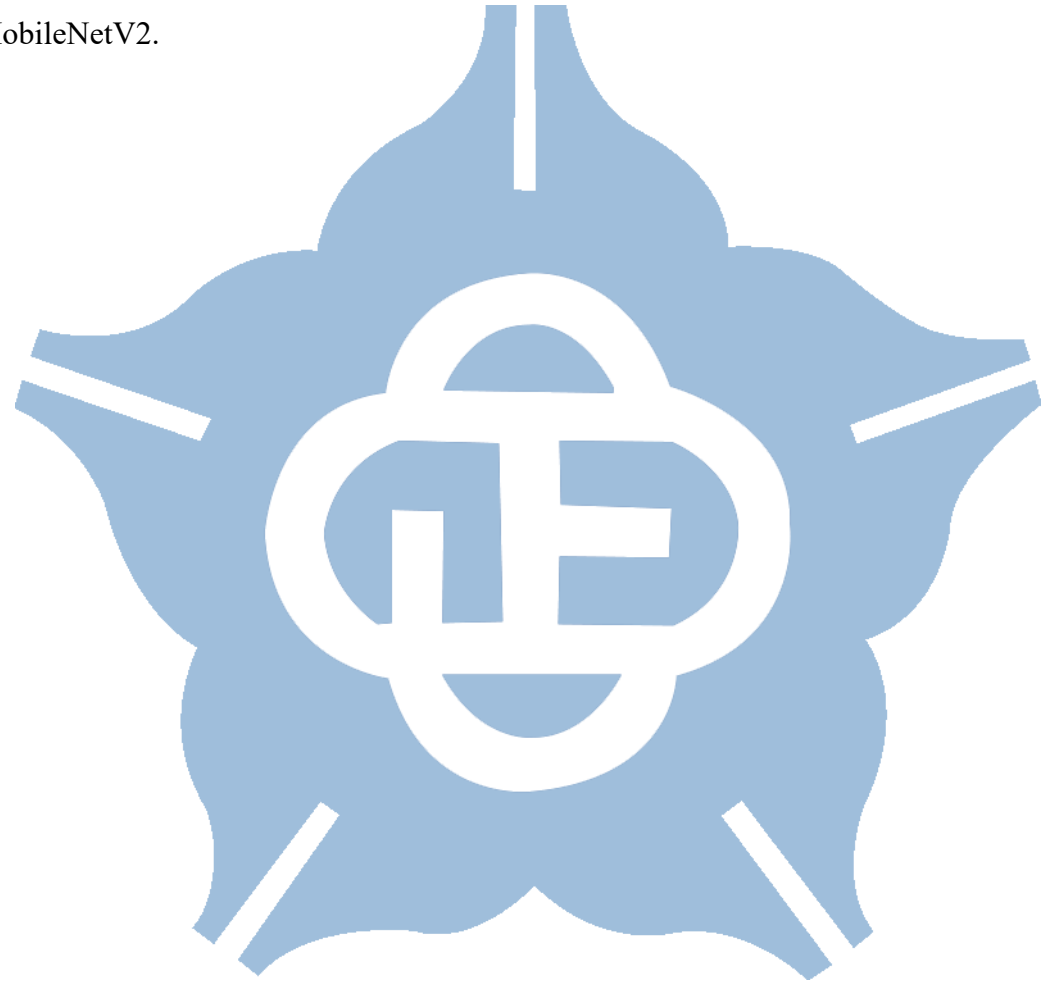
In Table 2.3, *t* denotes the expansion factor used in bottleneck layers, which controls the width of the intermediate feature map. *c* indicates the number of output channels for each block, while *n* specifies how many times the block is repeated. *s* represents the stride, determining the downsampling rate of the input feature map.

Table 2.3 Modified MobileNetV2 architecture.

Input	**Operator**	***t***	***c***	***n***	***s***
$100^2 \times 3$	conv2d	-	32	1	2
$50^2 \times 32$	bottleneck	1	16	1	1
$50^2 \times 16$	bottleneck	4	24	2	2
$25^2 \times 24$	bottleneck	4	32	1	1
$25^2 \times 32$	bottleneck	4	64	2	2
$13^2 \times 64$	bottleneck	4	96	1	1
$13^2 \times 96$	bottleneck	4	128	2	2
$7^2 \times 128$	bottleneck	4	192	1	1
$7^2 \times 192$	conv2d 1×1	-	728	1	1
$7^2 \times 728$	avgpool 7×7	-	-	1	-
$1 \times 1 \times 728$	conv2d 1×1	-	7	-	

The customized design reflects the adjustments made throughout the workflow, including reducing the number of blocks, scaling the channel widths, and selectively controlling spatial resolution to suit the 100×100 input size. These modifications result in a highly efficient architecture, which is then enhanced through pre-training and the

application of EAC. The EAC method enforces consistency between the attention maps of original and horizontally flipped images while incorporating random erasing as a data augmentation strategy. This dual mechanism encourages the network to capture globally relevant features and mitigates over-reliance on local details, thereby improving robustness to noisy labels and leading to a significant accuracy of 86.47% with only 659k parameters and 68M MACs, which is less than one-third of the original MobileNetV2.



2.3 Quantization Strategy and Experimental Results

To enhance the inference performance of the model on FPGA, quantization is required to avoid the complexity of floating-point operations and the significant memory burden they impose. This study employs integer quantization to 8 bits, which, compared to fixed-point quantization, not only better preserves accuracy but also simplifies hardware design by using a unified bit width throughout the system. In this work, all quantization is performed using PTQ, ensuring that the trained floating-point model can be efficiently converted to an integer-only representation without additional retraining.

As discussed in Section 1.4, integer quantization involves several key configuration considerations. Following the guidelines outlined in [27], which sets activations to per-layer symmetric quantization, and evaluates various configurations for the weights. The configuration is adopted because activations typically exhibit narrower and more consistent dynamic ranges across channels, making per-layer quantization sufficient to maintain accuracy while simplifying implementation. Additionally, symmetric quantization eliminates the need to store zero-point offsets for each layer, reducing hardware complexity and memory usage, which is particularly beneficial for edge devices such as FPGAs.

Table 2.4 reports the impact of various weight quantization settings on model accuracy. It is evident that the per-channel symmetric quantization configuration yields the highest accuracy at 86.25%, closely approaching the original floating-point model's accuracy of 86.47%. Per-channel quantization allows finer-grained scaling, effectively accommodating the variability between different filter channels, which is often critical in deep convolutional architectures. Based on these results, the final quantization

strategy in this work adopts per-channel symmetric quantization for weights and per-layer symmetric quantization for activations to balance accuracy and hardware efficiency.

Interestingly, in this study, symmetric quantization achieves higher accuracy than asymmetric quantization, even though asymmetric scaling is generally expected to offer finer numerical representation. It could be attributed to the relatively uniform distribution of weight values in the model, reducing the advantage typically offered by asymmetric scaling. Given these findings, symmetric quantization is clearly the preferred choice, not only for its superior performance but also because it simplifies hardware implementation.

Table 2.4 Model accuracy under different weight quantization settings.

Activations	Per Layer Symmetric			
Weight	Per Channel Symmetric	Per Layer Symmetric	Per Channel Asymmetric	Per Layer Asymmetric
Accuracy (%)	86.25	83.93	85.72	84.97

Overall, this quantization strategy demonstrates that the model achieves substantial compression with minimal accuracy loss, providing a solid foundation for efficient FPGA implementation.

Table 2.5 provides a comparative analysis of other lightweight CNN methods evaluated on the RAF-DB dataset. References [16] and [17] introduced simplified CNN architectures, namely A-MobileNet and Custom Lightweight CNN-based Model (CLCM), respectively. A-MobileNet combines center loss with CBAM, whereas CLCM utilizes basic data augmentation techniques. Although these two models have fewer parameters than MobileNet, they exhibit relatively lower accuracy, each

achieving an accuracy of 84% on the RAF-DB dataset.

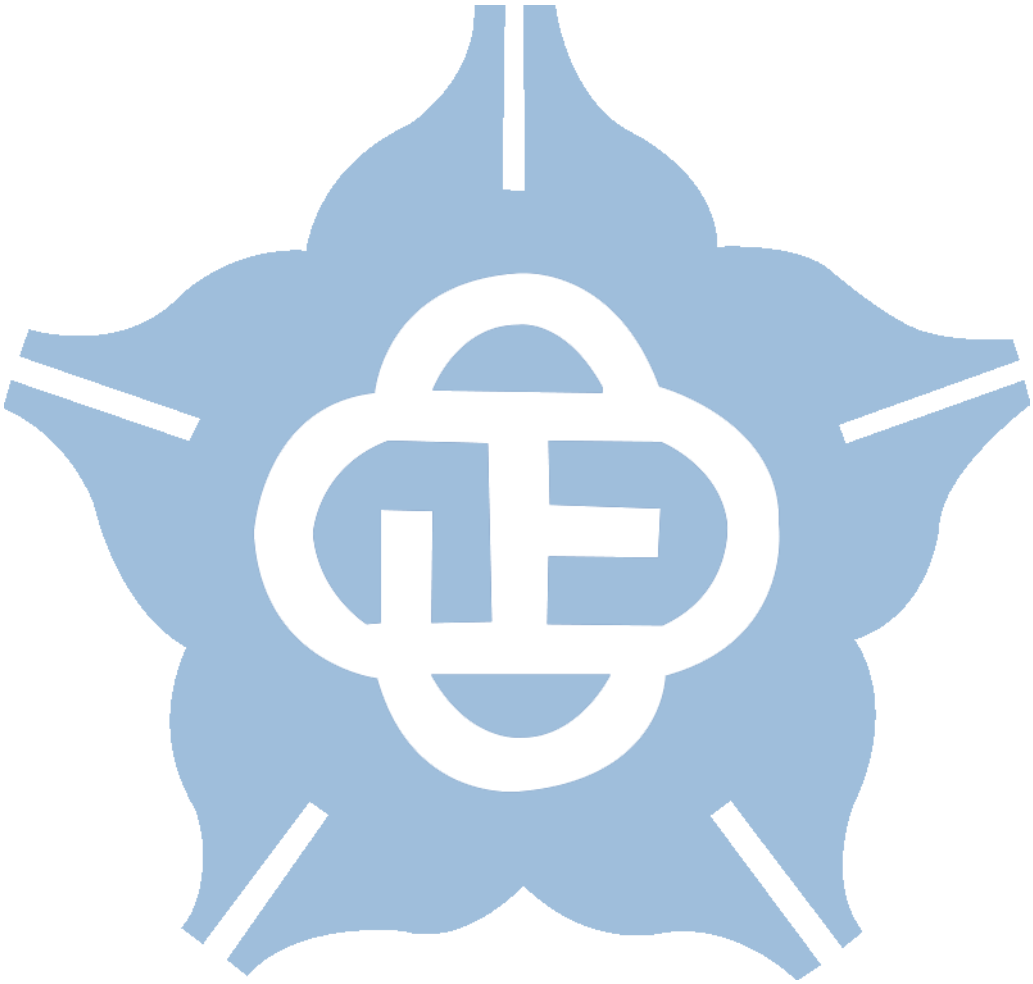
In [18], ZenNAS was employed to identify a high-performance micro-architecture, and a SA mechanism was introduced for feature refinement, improving accuracy to 88.53%. In [20], SCN was utilized, achieving an accuracy of 87.03%. However, both approaches still present opportunities for further reduction in computational cost and parameter size.

Table 2.5 Comparison with other lightweight CNN methods on RAF-DB.

	[16] Alexandria Engineering' 22	**[17] IEEE Access' 24**	**[18] ICCAS' 24**	**[20] CVPR' 20**	**This work**
Resolutions	100×100	224×224	224×224	224×224	100×100
Precision	FP32	FP32	FP32	FP32	INT8
Architecture	MobileNetV1	CLCM	ZenNet-M-SA	ResNet-18	MobileNetV2
Accuracy (%)	84.49	84	88.53	87.03	86.31
Parameters Size (MB)	13.6	9.16	3.85	42.65	0.66
FLOPs	N/A	599M	400M	1G	68M

The proposed method achieves a notable accuracy of 86.31% while processing on 100×100 input images and using 8-bit integer quantization. Despite using a compact model with only 0.66 MB of parameters and 68 million FLOPs, it delivers comparable or superior accuracy to several state-of-the-art methods that rely on full-precision (FP32) computations. In summary, the quantized MobileNetV2 presented in this work demonstrates an excellent balance between recognition accuracy, model efficiency, and

hardware friendliness, demonstrating strong potential for practical real-time FER systems on constrained platforms.



Chapter 3 Hardware Implementation

3.1 Scale Alignment for Integer-Only Residual Addition in CNN Accelerators

Implementing quantized CNNs on FPGA platforms necessitates performing all computations using integer arithmetic to satisfy strict constraints on power consumption, latency, and hardware resources. However, architectures incorporating residual connections, such as MobileNetV2, pose unique challenges. Specifically, outputs from different branches generally possess distinct quantization scaling factors, thus preventing straightforward integer addition.

A practical method to perform residual addition between two quantized tensors with differing scales involves requantizing only one of the branches (typically the input or residual path). This adjustment allows both tensors to be represented using a common scale factor prior to addition. Because the residual path continuously preserves activation distributions across layers, the output activations generally align with the residual branch scale. This approach maintains internal consistency within the model feature representations. Modifying only a single operand ensures the method remains mathematically valid and suitable for integer quantization procedures.

Although this strategy enables addition between differently scaled tensors, it does not eliminate the underlying computational overhead. Specifically, rescaling integer values involves multiplying by a scale factor, which typically requires multiplication, addition, and rounding operations. On FPGA platforms, such arithmetic increases circuit complexity and resource usage, making it less favorable in systems with tight latency and power constraints. Therefore, the cost of requantization remains a key issue

for efficient hardware implementation.

To reduce the computational cost of residual addition, prior work [33] proposed a scale factor alignment method by setting identical scale factors for the main and residual branch activations. This alignment facilitates residual addition using straightforward and efficient integer arithmetic. However, even minor adjustments to the original scale factors may introduce slight deviations in numerical precision and decrease model accuracy. Therefore, in this work, repeated application of this alignment method to residual blocks resulted in a modest accuracy drop from 86.25% to 86.18%. However, considering the substantial reductions in hardware complexity and computational cost, this slight accuracy degradation is considered acceptable.

Building on the concept of scale alignment, this work proposes a refined, hardware-friendly strategy that sets the input quantization scale to a value computable via simple operations, eliminating the need for floating-point arithmetic. A naive approach might be to simply round the scale factor to a power-of-two value, enabling bit-shifting as the sole operation. While attractive from a hardware design perspective, power-of-two often causes significant deviation from the original floating-point scale, which can result in large quantization errors and degrade model accuracy.

To mitigate this degradation, this work employs integer multiplication followed by a shift operation, as shown in Equation 3.1. On the left-hand side of the equation, the input data x is quantized by dividing it by the input floating-point scale factor S_i and adding a zero point Z_i. On the right-hand side, this operation is approximated by an integer multiplication with a constant N followed by a right shift of R bits. In practice, R is first selected according to hardware efficiency requirements, and N is then computed accordingly. The same R and N values applied to all input data preprocessing to maintain uniform scaling across the system, enabling fine-grained scaling with minimal deviation from the original numerical range.

$$\frac{x}{S_i} + Z_i := x * M \gg R + Z_i \tag{3.1}$$

To further reduce computational cost, experimental evaluations were conducted using various values of R, which control the bit-width of the integer multiplier used in the scale approximation. Results presented in Table 3.1 indicate that with R = 4, the model maintains its original accuracy while demanding minimal hardware resources.

In this context, the parameter R is chosen such that $x \times N >> R$ can effectively approximate x/S_i. When R is set to 4, the input scale obtained from the quantization result is 0.007808687165379524. Considering that the input data preprocessing in this work applies a normalization factor of 1/256, the effective scale S_i can be computed by combining the input scale with the preprocessing normalization, yielding S_i = $3.050268421 \times 10^{-5}$. Based on this value, N is determined as N = round($(1 << R)/S_i$), resulting in N = 2049. Consequently, the quantization process for the input data consists of multiplying the input by 2049, right-shifting the result by 4 bits, and then adding an zero point of 128, thereby completing the quantization procedure.

This configuration enables inference entirely without floating-point computation. As a result, the proposed strategy ensures that all operations, including residual additions, can be executed purely with integer arithmetic, leading to a highly efficient and fully quantized CNN suitable for real-time FPGA implementation.

Table 3.1 The accuracy of using different R factors.

R	**Accuracy**
5	86.18%
4	**86.18%**
3	85.95%
2	85.85%

3.2 Optimization of Quantization Multipliers in Integer CNN Inference

Following the alignment of quantization scales in residual connections, further improvements in efficiency can be achieved by optimizing the bit width configuration of the multiplier used in requantization. Specifically, the multiplier M_0 in Equation 1.4 determines the scaling factor during the requantization step, significantly influencing the computational overhead and hardware complexity of integer inference pipelines.

For example, if $M = 0.123456$ and it is multiplied by the integer 100, the result, when rounded to the nearest integer, is 12. If M is decomposed into a shift value k and a 32-bit M_0, then $k = 3$ and $M_0 = 0.(\text{FCD67FD4})_{16}$; the multiplication after decomposition still yields 12. When the bit width of M_0 is reduced to 8 bits, k remains 3 while M_0 becomes $0.(\text{FC})_{16}$, yet the computation still produces 12. This example illustrates that the precision of M_0 can be reduced to lower computational cost without affecting the correctness of the result.

This section focuses on analyzing the relationship between the bit width of M_0 and the resulting classification accuracy. A series of controlled experiments were conducted using various bit-width settings ranging from 12 bits down to 4 bits. Results presented in Table 3.2 demonstrate that bit widths between 12 and 5 yield stable accuracy, with the 5-bit configuration maintaining 86.31%, which is comparable to or slightly better than higher-bit settings. A further reduction to 4 bits, however, leads to a substantial drop in accuracy, indicating a practical lower bound for acceptable performance.

Table 3.2 Comparison of accuracy across different bit width of M_0.

bit width of M_0	**Accuracy**
12	86.31%

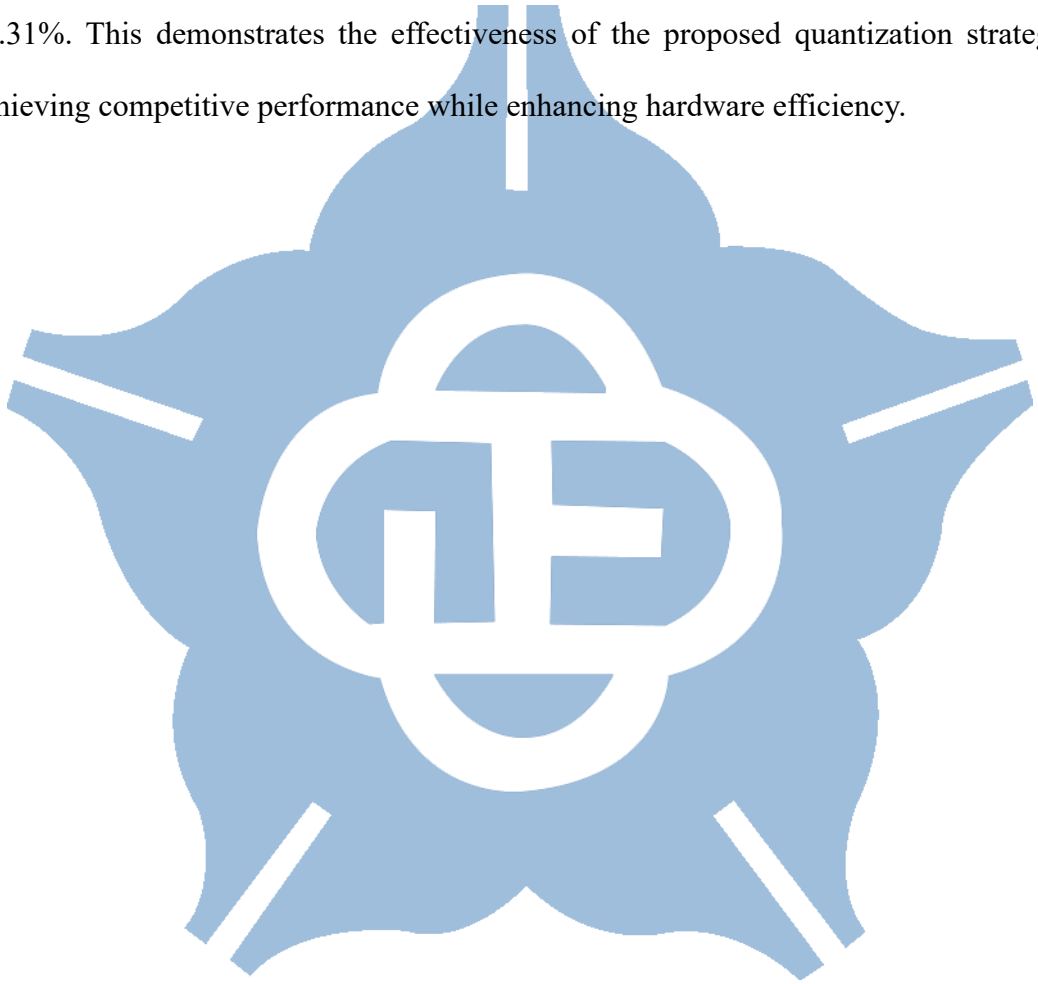
11	86.51%
10	85.95%
9	86.28%
8	85.98%
7	86.25%
6	85.66%
5	**86.31%**
4	76.37%

In contrast to the 32-bit multiplier used in the baseline method described in [26], the proposed 5-bit configuration significantly reduces computational cost. The bit width of M_0 primarily influences the precision of representing fractional scaling values. However, the actual scaling effect is more directly influenced by the shift value k, which determines the dynamic range and effective quantization step size. Consequently, reducing the bit width of M_0 has minimal impact on functional accuracy, as long as the shift value k provides sufficient resolution. This observation supports the use of lower-bit multipliers in hardware implementations without sacrificing performance.

Table 3.3 Progressive quantization modification with accuracy results.

Version	**Modification Description**	**Accuracy**
V0	Floating Model	86.47%
V1	Activations: per layer symmetric Weight: per channel symmetric	86.25%
V2	Adjust the shortcut connection scale and R of the quantization stub	86.18%
Final	Adjust the bit width of M_0	86.31%

A summary of all progressive quantization modifications and their corresponding accuracy impacts is presented in Table 3.3. Table 3.3 summarizes the adjustments described throughout this and the preceding sections, showing how each step influences model performance. Starting from the floating-point baseline with an accuracy of 86.47%, successive modifications, including symmetric quantization, shortcut scale adjustment, and optimized bit-width selection, culminate in a final model accuracy of 86.31%. This demonstrates the effectiveness of the proposed quantization strategy, achieving competitive performance while enhancing hardware efficiency.



3.3 Hardware Architecture and Parallelism Design for MobileNetV2 Accelerators

The MobileNetV2 architecture comprises a series of inverted residual blocks, each consisting of a single depthwise convolution and two pointwise convolutions. To optimize hardware efficiency, the dataflow design must be precisely tailored to match the structural characteristics of these inverted residual blocks. This work adopts the design strategy proposed in [34], where RAM content is overwritten using depthwise convolution and residual connections to minimize memory consumption. Because both operations preserve spatial and channel dimensions, intermediate data can be read from RAM, processed, and then written back to the same location, enabling efficient memory reuse. Moreover, [34] employs a single shared convolution engine to sequentially execute both depthwise and pointwise operations, thereby reducing hardware overhead compared to designs that allocate separate units for each type of convolution.

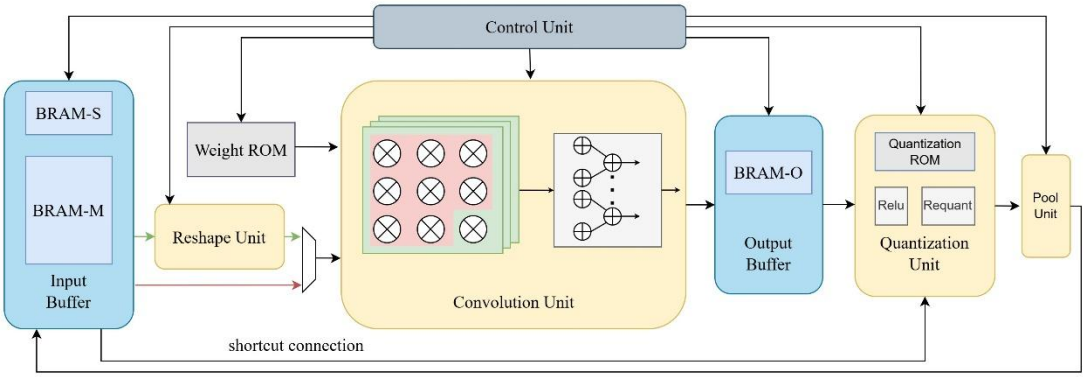


Figure 3.1 Overview of the MobileNetV2 accelerator.

Figure 3.1 illustrates the proposed MobileNetV2 accelerator architecture. Two distinct RAM units, block RAM-S (BRAM-S) and block RAM-M (BRAM-M), are used within the input buffer to store the main feature maps. These RAMs alternately store the output feature maps. Given that the MobileNetV2 configuration adopted in

this design uses an expansion factor of 4, BRAM-M is configured to be four times the size of BRAM-S. Accordingly, BRAM-M is designated for storing the expanded feature maps, while the remaining feature maps are stored in BRAM-S. In addition, BRAM-S is also utilized to store the residual connection data. Benefiting from the structural regularity of inverted residual blocks, this dual-RAM scheme effectively accommodates all required feature maps.

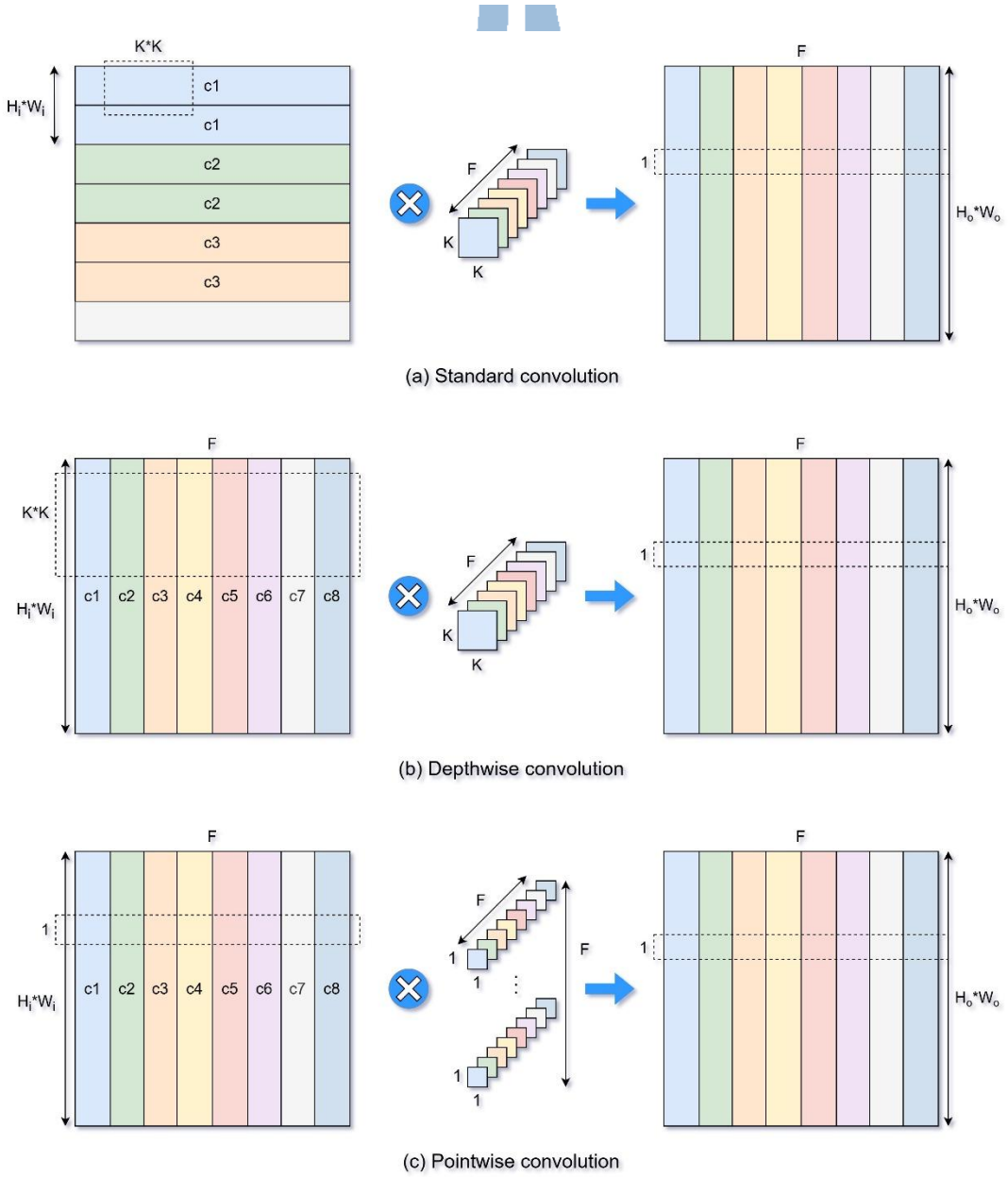


Figure 3.2 Different convolution operations and the memory dataflows.

In the proposed MobileNetV2 accelerator, the computational parallelism of the

filter dimension F, is set to 8, and K is set to 3. Accordingly, the convolution unit consists of eight parallel multiplication kernels. Each kernel contains 9 multipliers, resulting in a total of 72 multipliers. The green data path represents both the depthwise convolution and standard convolution operations. In this path, the reshape unit organizes the data into a 3×3 kernel size before sending it to the convolution unit for processing. In contrast, the red data path corresponds to the pointwise convolution, in which the data is directly fed into the convolution unit. Each channel is processed with 8 filter groups simultaneously, enabling 64 parallel multiplications.

After computation, the output data is transferred to the output buffer for partial summation. Once all filters have completed their operations, the accumulated results are passed to the quantization unit for quantization processing and subsequently routed back to the input buffer. If the current layer requires a shortcut operation, the quantized results are directly added to the values stored in BRAM-S.

The operational structure of the convolution computation is depicted in Figure 3.2. All three types of convolution—standard, depthwise, and pointwise—utilize the same convolution unit, where c denotes the data of the corresponding channel. The figure also illustrates how data is arranged within memory. In MobileNetV2, the standard convolution layer appears only once and takes as input an image with three channels ($c1$, $c2$, $c3$) and the largest spatial dimensions. To prevent excessive BRAM depth requirements, input data used for standard convolution is stored sequentially in memory (BRAM-S), so in practice the size of BRAM-S is equal to the input size of the first bottleneck, $50^2 \times 32 \ / \ F$, while BRAM-M is four times larger. For the other two convolution types and the data arrangement of the remaining layers, the images are stored vertically in memory to facilitate parallel access to F data entries during each read.

In Figure 3.2(a), standard convolution involves sequentially arranged input data.

After being fetched from memory, the data is passed through a shift register, which feeds it step by step into the reshape unit to form a 3×3 kernel structure. This reshaped data is then processed by eight parallel filters, resulting in an 8×3×3 data stream. In contrast, Figure 3.2(b) and 3(c) illustrate a configuration where memory data is arranged vertically, meaning each memory access retrieves data from eight channels simultaneously for computation. After storing channels *c1* to *c8*, the subsequent channels starting from *c9* are stored in the same manner. Specifically, for depthwise convolution, the retrieved data from the eight channels is distributed to eight separate reshape units, each generating a 3×3 structure, thereby also forming an 8×3×3 data stream.

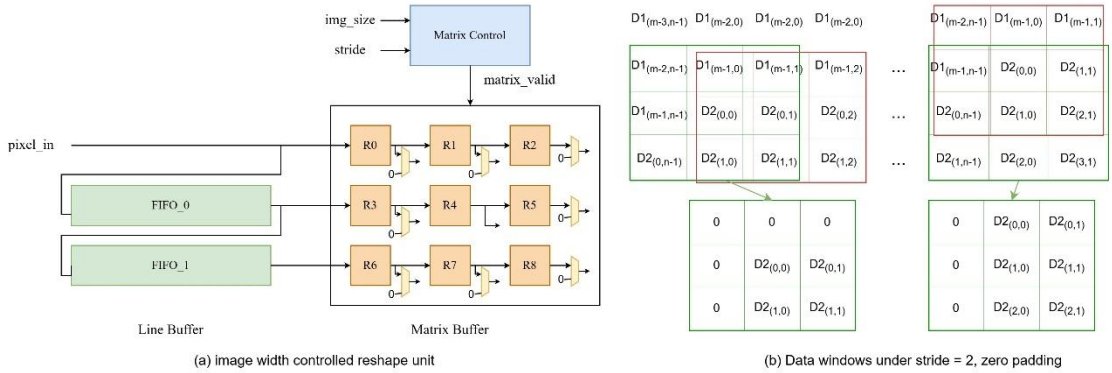
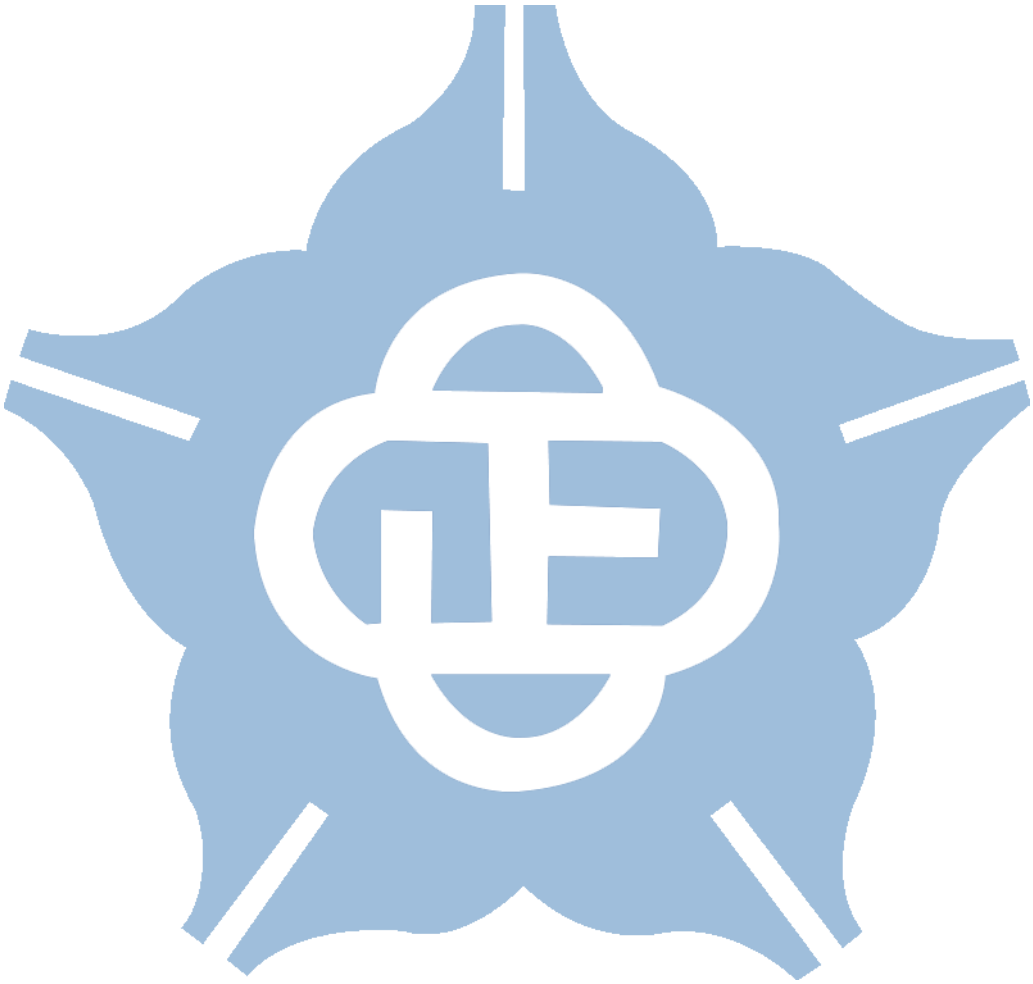


Figure 3.3 Architecture of the proposed reshape unit and the padding result.

Figure 3.3 shows the design of the proposed reshape unit. Conventional reshape units typically apply padding to images before feeding them into a FIFO buffer, allowing padded windows to be directly accessed at the FIFO output. However, this approach incurs additional clock cycles due to the propagation of padding values. For an input feature map of size H×W, this method would require (H+2)(W+2) cycles. In contrast, the proposed reshape unit leverages additional image size and stride metadata to instruct the matrix controller on when a window is valid and whether padding is necessary at the output. As demonstrated in Figure 3.3(b), which illustrates the output of the reshape unit under a stride setting of 2, invalid windows (red) are skipped by

suppressing the output of rows or columns at stride intervals, while valid windows (green) are padded at the edges based on image size. Using this strategy, the reshape unit requires only H×W cycles to transmit an image, eliminating the need for additional padding-related cycles.

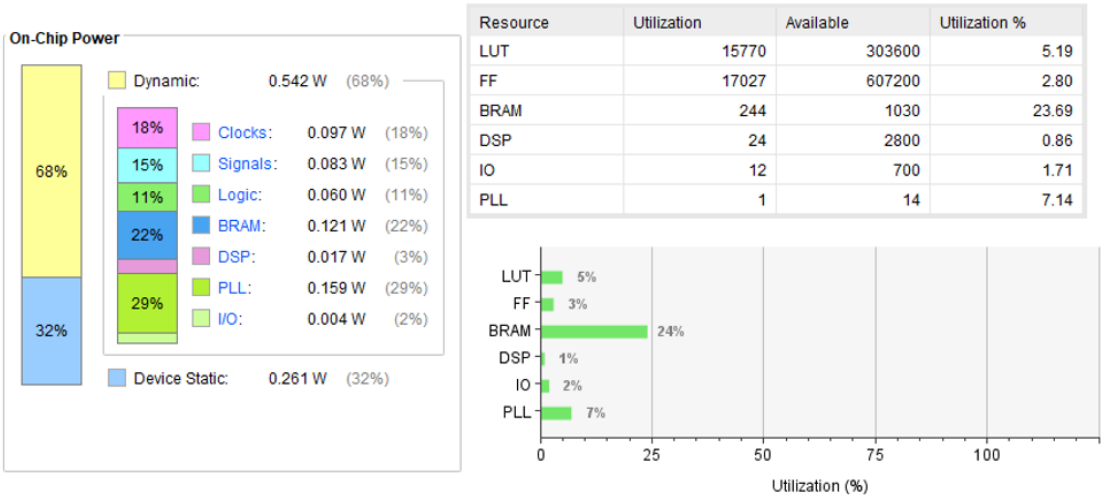


3.4 FPGA Experiment Result and Comparison

To evaluate the real-world performance and resource efficiency of the proposed MobileNetV2 accelerator, this section presents detailed experimental results obtained from FPGA implementation. The analysis includes comprehensive hardware metrics such as lookup tables (LUTs), flip-flops (FFs), digital signal processors (DSPs), block memory (BRAM), and maximum operating frequency. In addition, latency and throughput measurements are conducted to assess the system's real-time processing capability. To achieve real-time processing, a target of over 20 FPS is set. A comparison table is also provided to benchmark the proposed design against existing state-of-the-art FPGA-based facial expression recognition implementations.

In this work, the FER CNN accelerator was implemented on a Virtex-7 VC707 evaluation board using AMD Vivado 2024.2. The proposed accelerator consumes 560,000 cycles for the first five bottlenecks and 800,000 cycles for the last five bottlenecks, totaling 1.36785 million cycles, achieving 73 FPS at a clock frequency of 100 MHz. The associated power consumption and resource utilization metrics are presented in Figure 3.4. The circuit operates at 100 MHz, with its critical path located in the memory read stage. Among all hardware resources, BRAM has the highest utilization, primarily due to the storage of weights. Specifically, the weight ROM and quantization ROM, which stores M_0, shift values, and other quantization-related constants, consume 158 and 8.5 BRAM units, respectively, collectively accounting for over 70% of the overall BRAM usage. Each BRAM unit is 36 Kb, and Vivado also allows the use of half units with a capacity of 18 Kb. Additionally, this work also demonstrates operation at 50 MHz, achieving 36 FPS while consuming even less power. In the architecture shown in Figure 3.1, the memory demands for input and output buffers are minimal. Moreover, due to the adoption of a single convolution engine, the

utilization rates of LUT and DSP resources remain comparatively low.



Resource	Utilization	Available	Utilization %
LUT	15770	303600	5.19
FF	17027	607200	2.80
BRAM	244	1030	23.69
DSP	24	2800	0.86
IO	12	700	1.71
PLL	1	14	7.14

Figure 3.4 Power and resource utilization.

Table 3.4 presents a comparative analysis of the presented design against other FPGA-based FER implementations. Reference [30] employs multi-threading to perform face detection and FER tasks concurrently. However, it does not focus on optimizing FER accuracy, achieving only 67.4% on the FER-2013 dataset. This raises concerns regarding its practicality for real-world deployment. In [3], LBP are used for feature extraction, feeding the output into an ANN. The computational simplicity of LBP enables a frame rate of 27.5 FPS. However, in the current era dominated by deep learning, handcrafted feature extraction methods often underperform compared to end-to-end learning models. This approach achieves only 88% accuracy, which is noticeably lower than other methods reported on the JAFFE dataset. Consequently, its performance in facial expression recognition appears suboptimal, particularly when extended to more complex FER datasets, thus limiting its robustness and scalability.

Reference [31] applies INT8 quantization and leverages LLTQ to mitigate the accuracy degradation caused by quantization, achieving an 86.58% accuracy on the FERPlus dataset. However, FERPlus images lack diversity in color, illumination, and context, potentially limiting their representativeness for real-world scenarios.

Additionally, due to the small image resolution (64×64), the system achieves only 10 FPS, indicating room for performance improvement.

Table 3.4 Comparison with prior methods.

	[30] CANDARW' 24	**[3] FRUCT' 24**	**[31] IEEE Access' 21**	**[32] CMC' 24**	**This work**	
Dataset	FER2013	JAFFE	FERPlus	RAF-DB	RAF-DB	
Resolutions	48×48	100×100	64×64	224×224	100×100	
Hardware information	Xilinx Zynq UltraScale+ + ARM Cortex-A53	Xilinx Zynq 7020 + ARM Cortex-A9	Xilinx ZC706 + ARM Cortex-A9	Xilinx KV260	Xilinx VC707	
Parameters	N/A	N/A	0.4M	11M	0.66M	
IOPs	N/A	N/A	28M	1.8G	68M	
Accuracy	67.4%	88%	86.58	77.15%	86.31	
Frequency [MHz]	400	166.67	250	650	100	50
LUT	27K	9K	5K	19K	15K	
FF	N/A	19K	7K	23K	17K	
BRAM	12	28	61	87	ROM: 166.5 RAM: 63.5	
DSP	118	11	49	0	24	
FPS	25	27.5	10	N/A	73	36
Power[W]	2.7	N/A	2.31	4	0.803	0.602

In [32], SCN is implemented on the RAF-DB dataset using a quantized ResNet-18 architecture, down to 3-bit precision for FPGA compatibility. Although inference speed is improved, the recognition accuracy remains approximately 77%. Moreover, this work utilizes images with a resolution of 224×224 pixels. From a hardware design perspective, the original image resolution of 100×100 should not be upscaled for processing, as this unnecessarily increases computation time without yielding proportional accuracy gains.

Compared to previous works, the proposed design demonstrates superior power efficiency and processing throughput. Moreover, this work is also evaluated on the NVIDIA Jetson AGX Orin platform for comparison. While the Jetson AGX Orin consumes over 15 W of power to achieve 15 FPS, the proposed FPGA-based approach achieves 73 FPS with only 0.8 W of power consumption. Given the complexity of the dataset and the relatively high image resolution, it achieves competitive and often superior accuracy, making it well-suited for real-world application.

Chapter 4 Conclusion and Future Works

4.1 Conclusion

This thesis proposes a fully integrated real-time FER framework implemented on an FPGA platform, employing an integer-only quantized CNN accelerator. MobileNetV2 serves as the backbone network, enabling the development of a lightweight and precise model customized explicitly for low-resolution inputs. Through architectural optimization and enhancement with noise-robust training techniques, such as the EAC method, the proposed model achieves high accuracy on the RAF-DB dataset.

To facilitate efficient implementation on resource-constrained FPGAs, an integer-only quantization strategy was adopted. This included per-channel symmetric quantization for weights and activations, as well as scale alignment techniques for residual connections. Further optimization of quantization multipliers enabled fully integer arithmetic with minimal accuracy loss, allowing the model to maintain 86.31% accuracy.

Finally, this work presents the design and implementation of a dedicated accelerator on a Xilinx VC707 FPGA evaluation board. The proposed hardware employs a highly efficient dataflow architecture, incorporating a single convolution engine and a dual RAM buffer strategy to maximize throughput while minimizing resource utilization. Experimental results demonstrate that the proposed hardware accelerator delivers outstanding performance in terms of accuracy, power consumption (0.803 W), and inference speed (73 FPS), outperforming several state-of-the-art designs in benchmark evaluations. This work confirms the practicality of implementing advanced FER systems on edge devices for real-time applications.

4.2 Future Work

The integer-only CNN accelerator developed in this study demonstrates strong performance in real-time facial expression recognition on FPGA, achieving both high accuracy and low latency. Nonetheless, several limitations remain. Misclassifications were observed when the system was tested with images collected from real-world sources. These errors are believed to stem from dataset bias, as the RAF-DB dataset predominantly consists of facial images from Western populations. When images from other ethnic groups like Eastern populations are input into the model trained solely on RAF-DB, recognition accuracy may degrade.

To improve generalization, future work should incorporate data from more diverse demographics and environments that reflect practical implementation scenarios. In addition, improving data quality through filtering and cleaning processes can reduce the influence of ambiguous images, thereby enhancing model robustness.

Regarding quantization, the current design employs an 8-bit integer format to reduce memory usage and hardware complexity. Future enhancements may consider the adoption of Quantization-Aware Training to retain accuracy while further reducing bit width, such as to 4-bit representations or mixed-precision formats. This approach can lower resource consumption without significant accuracy loss. Other hardware-friendly quantization methods, including learnable step-size technique, may also be explored to simplify arithmetic operations. These strategies contribute to further improving the efficiency and scalability of CNN accelerators in edge computing applications.

Reference

[1] Marian Stewart Bartlett, Gwen Littlewort, Ian Fasel, and Javier R. Movellan, "Real time face detection and facial expression recognition: development and applications to human computer interaction," *in Proceedings of Conference on Computer Vision and Pattern Recognition Workshop (CVPRW)*, Jun. 2003.

[2] Caifeng Shan, Shaogang Gong, and Peter W. McOwan, "Robust facial expression recognition using local binary patterns," *in Proceedings of IEEE International Conference on Image Processing (ICIP)*, Sep. 2005.

[3] Ahmed Chiheb Ammari, Lazhar Khriji, Medhat Awadalla, and Rami Al Hmouz, "Co-design of hardware and software for facial expression recognition using Xilinx Zynq SoC," *in Proceedings of Conference of Open Innovations Association (FRUCT)*, Nov. 2024, pp. 94-99.

[4] Daniel Octavian Melinte and Luige Vladareanu, "Facial expressions recognition for human-robot interaction using deep convolutional neural networks with rectified Adam optimizer," *Sensors*, vol. 20, no. 8, Art no. 2393, Apr. 2020.

[5] Ning Zhou, Renyu Liang, and Wenqian Shi, "A lightweight convolutional neural network for real-time facial expression detection," *IEEE Access*, vol. 9, pp. 5573-5584, Dec. 2020.

[6] Jiannan Yang, Tiantian Qian, Fan Zhang, and Samee U. Khan, "Real-time facial expression recognition based on edge computing," *IEEE Access*, vol. 9, pp. 76178-76190, May 2020.

[7] Tao Zhang, Minjie Liu, Tian Yuan, and Najla Al-Nabhan, "Emotion-aware and intelligent internet of medical things toward emotion recognition during COVID-19 pandemic," *IEEE Internet of Things Journal*, vol. 8, pp. 16002-16013, Nov. 2020.

[8] M. Lyons, S. Akamatsu, M. Kamachi, and J. Gyoba, “Coding facial expressions with Gabor wavelets,” *in Proceedings of IEEE International Conference on Automatic Face and Gesture Recognition (FG)*, Apr. 1998, pp. 200-205.

[9] Emad Barsoum, Cha Zhang, Cristian Canton Ferrer, and Zhengyou Zhang, “Training deep networks for facial expression recognition with crowd-sourced label distribution,” *in Proceedings of the 18th ACM International Conference on Multimodal Interaction (ICMI)*, Sep. 2016, pp. 279–283.

[10] Shan Li, Weihong Deng, and JunPing Du, “Reliable crowdsourcing and deep locality-preserving learning for expression recognition in the wild,” *in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jul. 2017, pp. 2584-2593.

[11] Ali Mollahosseini, Behzad Hasani, and Mohammad H. Mahoor, “AffectNet: a database for facial expression, valence, and arousal computing in the wild,” *IEEE Transactions on Affective Computing*, vol. 10, no. 1, pp. 18–31, Aug. 2017.

[12] Karen Simonyan and Andrew Zisserman, “Very deep convolutional networks for large-scale image recognition, arXiv:1409.1556v6 [cs.CV],” *arXiv.org*, Apr. 2015.

[13] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam, “MobileNets: efficient convolutional neural networks for mobile vision applications, arXiv:1704.04861v1 [cs.CV],” *arXiv.org*, Apr. 2017.

[14] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen, “MobileNetV2: inverted residuals and linear bottlenecks,” *in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2018, pp. 4510-4520.

[15] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc

V. Le, and Hartwig Adam, “Searching for MobileNetV3,” *in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, May 2019, pp. 1314-1324.

[16] Yahui Nana, Jianguo Jua, Qingyi Huaa, Haoming Zhanga, and Bo Wang, “A-MobileNet: an approach of facial expression recognition,” *Alexandria Engineering Journal*, vol. 61, no. 6, pp. 4435–4444, Jun. 2022.

[17] Mustafa Can Gursesli, Sara Lombardi, Mirko Duradoni, Leonardo Bocchi, Andrea Guazzini, and Antonio Lanata, “Facial emotion recognition (FER) through custom lightweight CNN model: performance evaluation in public datasets,” *IEEE Access*, vol. 12, pp. 45543-45559, Mar. 2024.

[18] Trang Nguyen, Hamza Ghulam Nabi, and Ji-Hyeong Han, “ZenNet-SA: An Efficient Lightweight Neural Network with Shuffle Attention for Facial Expression Recognition,” *in Proceedings of 2024 24th International Conference on Control, Automation and Systems (ICCAS)*, Dec. 2024, pp. 55-60.

[19] Amir Hossein Farzaneh and Xiaojun Qi, “Facial expression recognition in the wild via deep attentive center loss,” *in Proceedings of IEEE Winter Conference on Applications of Computer Vision (WACV)*, Jan. 2021, pp. 2401-2410.

[20] Kai Wang, Xiaojiang Peng, Jianfei Yang, Shijian Lu, and Yu Qiao, “Suppressing uncertainties for large-scale facial expression recognition,” *in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2020, pp. 6897-6906.

[21] Yuhang Zhang, Chengrui Wang, Xu Ling, and Weihong Deng, “Learn from all: erasing attention consistency for noisy label facial expression recognition,” *in Proceedings of European Conference on Computer Vision (ECCV)*, Jul. 2022, pp. 418-434.

[22] Dan Zeng, Zhiyuan Lin, Xiao Yan, Yuting Liu, Fei Wang, and Bo Tang, “Face2Exp:

combating data biases for facial expression recognition," *in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2022, pp. 20259-20268.

[23] Ce Zheng, Matias Mendieta, and Chen Chen, "Poster: a pyramid cross-fusion transformer network for facial expression recognition," *in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Aug. 2023, pp. 3146-3155.

[24] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio, "Binarized neural networks: training deep neural networks with weights and activations constrained to +1 or -1, arXiv:1602.02830v3 [cs.LG]," *arXiv.org*, Mar. 2016.

[25] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou, "DoReFa-Net: training low bitwidth convolutional neural networks with low bitwidth gradients, arXiv:1606.06160v3 [cs.NE]," *arXiv.org*, Feb. 2018.

[26] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2018, pp. 2705-2713.

[27] Raghuraman Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: a whitepaper, arXiv:1806.08342v1 [cs.LG]," *arXiv.org*, Jun. 2018.

[28] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan, "PACT: parameterized clipping activation for quantized neural networks, arXiv:1805.06085v2 [cs.CV]," *arXiv.org*, Jul. 2018.

[29] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha, "Learned step size quantization," *in Proceedings of International Conference on Learning Representations (ICLR)*, Apr. 2020.

[30] Takuto Ando and Yusuke Inoue, "Facial expression recognition system using DNN accelerator with multi-threading on FPGA," *in Proceedings of International Symposium on Computing and Networking Workshops (CANDARW)*, Nov. 2024, pp. 103-109.

[31] Jaemyung Kim, Jin-Ku Kang, and Yongwoo Kim, "A resource efficient integer-arithmetic-only FPGA-based CNN accelerator for real-time facial emotion recognition," *IEEE Access*, vol. 9, pp. 104367-104381, Jul. 2021.

[32] Fangzhou He, Ke Ding, Dingjiang Yan, Jie Li, Jiajun Wang, and Mingzhe Chen, "A novel quantization and model compression approach for hardware accelerators in edge computing," *Computers, Materials & Continua*, vol. 80, no. 2, pp. 3021-3045, Aug. 2024.

[33] Weixiong Jiang, Heng Yu, and Yajun Ha, "A high-throughput full-dataflow MobileNetv2 accelerator on edge FPGA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 5, pp. 1532-1545, May. 2023.

[34] Shun Yan, Zhengyan Liu, Yun Wang, Chenglong Zeng, Qiang Liu, and Bowen Cheng, "An FPGA-based MobileNet accelerator considering network structure characteristics," *in Proceedings of 2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, Aug. 2021, pp. 17-23.

[35] Chi Jin, Ruochun Jin, Kai Chen, and Yong Dou, "A community detection approach to cleaning extremely large face database," *Computational Intelligence and Neuroscience*, vol. 2018, Art no. 4512473, Apr. 2018.